AppliedMath

MDPI

Article

A Mathematical Framework for Four-Dimensional Chess: Extending Game Mechanics Through Higher-Dimensional Geometry

Rinaldi (Unciuleanu) Oana * and Costin-Gabriel Chiru *

Department of Computer Science, National University of Science and Technology POLITEHNICA Bucharest, 060042 Bucharest, Romania

* Correspondence: oana.rinaldi@stud.acs.upb.ro (R.O.); costin.chiru@upb.ro (C.-G.C.)

Abstract

This paper develops a rigorous mathematical and computational framework for four-dimensional chess defined on the discrete hypercubic lattice $\{1, \ldots, 8\}^4$. We formalize piece movement using displacement sets in $\mathbb{Z}^4$, define adjacency via the Chebyshev metric, and analyze the resulting move graphs for rooks, bishops, knights, queens, and kings. We establish exact mobility formulas, parity invariants, and connectivity properties, consolidating known product-graph results for rooks and kings while introducing a boundary-sensitive analysis of the four-dimensional knight verified by exhaustive enumeration. The mathematical framework is complemented by a fully implemented 4D chess engine and interactive visualization environment rendering all 64 (z,w)-slices of the hypercube simultaneously. The system supports full move legality, generalized special rules, multi-king checkmate detection, and reproducible state enumeration. Performance measurements and exploratory branching-factor estimates are obtained through reproducible random playouts using the publicly available implementation. We contextualize this ruleset within existing work on move graphs on $\mathbb{Z}_n^m$, higher-dimensional leapers, spectral properties of grid graphs, toroidal analogs, and multidimensional visualization. Exploratory qualitative feedback (N = 18) is included to examine whether the visualization design is interpretable and navigable in practice, providing feasibility-oriented observations on how slice-based 4D projection and layered board rendering are perceived by non-expert users in an exploratory context. Together, the mathematical results, implemented engine, and visualization form a coherent foundation for the study of strategy, complexity, and human interaction in four-dimensional game systems. The framework provides a basis for future investigations into spectral analysis of move graphs, symmetry-aware search, hierarchical planning, and educational applications in high-dimensional geometry.

Keywords: four-dimensional geometry; combinatorial game theory; mathematical modeling; graph theory; projection systems; game mechanics; higher-dimensional visualization

Check for updates

Academic Editors: Thomas Woolley and Chaudry Masood Khalique

Received: 30 December 2025
Revised: 5 February 2026
Accepted: 5 March 2026
Published: 17 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The extension of classical games into higher-dimensional settings has long served as a source of mathematical insight, cognitive challenge, and conceptual experimentation. From Abbott's *Flatland* [1] to modern explorations of multidimensional geometry [2,3], the idea of reasoning within dimensions beyond three has informed perspectives in topology, combinatorics, visualization, and theoretical computer science. Chess, with its rich structural

constraints, well-defined move semantics, and deeply studied combinatorial landscape, provides an especially suitable testbed for such dimensional generalization.

While several structural properties studied here, such as rook and king move graphs arising as Cartesian or strong products, are known consequences of classical product-graph theory, the contribution of this work lies in their systematic integration into a single four-dimensional chess framework that is fully formalized, computationally implemented, and empirically validated. Rather than proposing isolated observations about individual pieces, we provide a coherent Z^4 displacement-based ruleset, explicit boundary-sensitive mobility analysis, multi-king legality semantics, and a reproducible engine and visualization that together establish a concrete foundation for studying higher-dimensional game systems.

We emphasize that this work is intended as a rigorously specified and reproducible baseline for four-dimensional chess on a fixed finite lattice; extensions to general $[n]^d$ boards or to spectral and isoperimetric analyses are natural next steps but lie beyond the scope of the present study.

In this work, we develop a complete, formal, and computationally implemented framework for *four-dimensional chess* on the discrete hypercubic lattice $B = \{1, \ldots, 8\}^4 \subset \mathbb{Z}^4$, where each coordinate axis defines an independent spatial dimension. Our goals are threefold:

Mathematical formalization

We generalize classical chess movements to four dimensions using vector displacement sets, define adjacency via the Chebyshev metric, and analyze the resulting *move graphs* for standard pieces. We present a corrected and uniform rook mobility formula, the proof that the 4D rook graph is connected with diameter 4, a constructive proof that bishop movement connects each parity class, and a boundary-sensitive classification of knight degrees verified by exhaustive enumeration.

Algorithmic and ruleset design

We extend special rules—castling, promotion, en passant—and generalize check, stalemate, and checkmate to positions with multiple kings, as necessitated by high-density initial configurations. All rules are implemented and validated within a complete engine, including pseudo-legal and legal move filtering, attack-map construction, repetition detection, and 50-move rule handling.

Visualization and interaction in 4D space

Humans cannot directly perceive four spatial dimensions, making explicit projection or slicing strategies necessary for interaction with 4D structures. Accordingly, beyond formalizing movement and rules, this work treats visualization as a functional component of the framework: a representation that preserves adjacency, directionality, and cross-dimensional relationships sufficiently to support gameplay and inspection of the formal move geometry.

We first situate the work within the existing literature, then develop the mathematical foundations of 4D chess, followed by implementation details, computational analysis, and exploratory evaluation.

1.1. Motivation and Theoretical Context

The mathematical appeal of higher-dimensional chess stems in part from its relation to the study of *move graphs* on $\mathbb{Z}_n^m$, a setting where displacement sets define graph edges and many algebraic and geometric techniques apply [4]. Rook, bishop, and knight moves extend naturally to four dimensions, producing graphs with distinctive connectivity, parity, and mobility properties. Our analysis contributes explicit formulas, invariants, and constructive

proofs that fit into the broader literature on grid graphs, rook graphs, toroidal models, and leaper graphs.

From a complexity perspective, classical results show that generalized chess on $n \times n$ boards is EXPTIME-complete [5], and "mate-in-k" formulations remain intractable. While our setting is fixed at an $8 \times 8 \times 8 \times 8$ board, the increase from 64 squares to 4096 induces dramatic growth in branching factors, state-space size, and search-tree expansion—paralleling the combinatorial explosion Shannon emphasized in his foundational work on chess computation [6]. Our empirical branching-factor measurements (mean ≈ 74) reflect this complexity expansion.

On the human–computer interaction side, the design of visualization systems capable of conveying 4D relationships is itself a nontrivial challenge. Classical principles from Banchoff [2] and Ware [7] emphasize slice-based, projection-based, and task-oriented views as means of understanding higher-dimensional geometry. Quaternion-based navigation provides smooth and perceptually stable camera control [8], while layered and partially transparent representations mitigate occlusion in dense 3D scenes [9].

1.2. Contributions

While several results are natural extensions of known 2D and 3D cases, we present them here in a unified, fully implemented 4D framework that enables direct computational validation and applied investigation.

This paper provides the following contributions:

Mathematical Contributions

1. **Uniform rook mobility**: a corrected, elementary proof that all squares yield mobility exactly 28, where mobility denotes the number of squares reachable by a single legal move.
2. **Rook graph connectivity**: proof that the graph is connected with diameter 4.
3. **Bishop invariants and connectivity**: bishop moves preserve coordinate-sum parity; each parity class is provably connected via constructive diagonal sequences that handle boundary detours.
4. **Knight mobility**: complete characterization of interior and boundary-sensitive degrees with exhaustive enumeration across all 4096 squares.

Algorithmic and Ruleset Contributions

1. **Comprehensive 4D chess engine** with
   - full move generation,
   - attack-map construction,
   - legality filtering across all kings,
   - generalized castling and en passant rules,
   - repetition and 50-move rule detection,
   - Zobrist-style hashing extended to 4D: constant-time hashing of 4D board states for fast position comparison and reproducible state enumeration.
2. **Complete reproducibility** of enumeration and empirical results through publicly available code and online engine.
3. Formalization of multi-king checkmate: a ruleset that generalizes classical attack and legality concepts to multiple royal pieces.

Visualization and HCI Contributions

1. A 64-slice 3D visualization of the 4D chess hypercube, rendering all $(z\prime w)$-slices simultaneously.
2. Interactive camera navigation via quaternions, layered transparency, and consistent coordinate annotation.

The novelty of this work lies not in redefining individual chess moves per se, but in providing a mathematically rigorous, computationally complete, and visually accessible integration of four-dimensional move geometry, rules, and gameplay within a single coherent framework.

Beyond its role as a playable game, the proposed four-dimensional chess framework serves as a controlled experimental environment for studying reasoning, planning, and visualization in high-dimensional discrete spaces. While the full game state involves a large number of pieces and is not intended for unaided human play, the framework is well-suited for the development and testing of advanced search algorithms, symmetry-aware evaluation methods, and learning-based agents operating on structured high-dimensional graphs. In addition, the interactive visualization component supports cognitive exploration and conceptual learning of four-dimensional geometry, making the system relevant both as a computational testbed and as an educational tool.

1.3. Paper Structure

Section 2 reviews related work on generalized chess, move graphs on higher-dimensional lattices, combinatorial game complexity, and visualization of multidimensional structures. Section 3 develops the mathematical framework for four-dimensional chess, including the lattice model, adjacency, piece displacement sets, mobility formulas, parity invariants, and connectivity properties of the resulting move graphs. Section 4 describes the full system implementation, covering engine architecture, move generation, special rules, multi-king legality, visualization design, symmetry constraints, and the computational complexity of move generation. Section 5 presents computational results and exploratory analyses, including branching-factor measurements, qualitative strategic observations, endgame demonstrations, complexity-theoretic context, and exploratory user feedback. Section 6 concludes with a discussion of implications, limitations of the present work, and directions for future research.

2. Related Work

Research relevant to four-dimensional chess spans several distinct areas: the computational complexity of generalized board games, algebraic and combinatorial analysis of move graphs on high-dimensional lattices, leaper graph theory, spectral graph properties of grid-based move sets, toroidal analogs on $\mathbb{Z}_n^m$, and visualization techniques for multidimensional structures. This section situates our work within these domains and clarifies its contributions relative to the existing literature.

2.1. Generalized Chess and Computational Complexity

The computational difficulty of chess-like games and their many variants has been studied since Zermelo's early application of game theory to chess [10], with extensive cataloging and classification of chess variants documented by Pritchard [11]. Shannon's seminal analysis demonstrated that even classical 2D chess has a game-tree complexity on the order of 10^{120}, emphasizing the exponential dependence on the branching factor [6]. When the board size or rules are generalized, the problem becomes dramatically harder: Fraenkel and Lichtenstein proved that computing a perfect strategy for $n \times n$ chess requires time exponential in n [5]. Further refinements in the theory of algorithmic hardness for combinatorial games by Hearn and Demaine show that generalized chess is EXPTIME-complete, situating chess-like games within the broader mathematical study of combinatorial games [12].

These works do not directly treat additional spatial dimensions, but the complexity drivers—state-space size and move branching—scale naturally with dimension. In our 4D 8^4-board setting, even fixed-size positions exhibit branching factors often exceed-

ing 70 moves, indicating substantial growth relative to 2D. While we do not claim any new complexity-theoretic results, our engine-based measurements support the conclusion that 4D chess inherits the practical intractability typical of generalized grid-based games. Foundational perspectives on the mathematical analysis of games, including structural, combinatorial, and symmetry-based reasoning on board games, are presented in the classical work of Conway, Guy, and Berlekamp [13].

2.2. Move Graphs on $\mathbb{Z}_n^m$: Product-Graph Structure

Many chess pieces give rise to product graphs when formalized as move graphs on discrete lattices. Let $B = \{1, \ldots, 8\}^4 \subset \mathbb{Z}^4$ and consider the graph whose vertices are lattice points and whose edges correspond to legal moves of a given piece on an empty board.

Rook graphs and Hamming graphs

The rook move graph on B is exactly the Hamming graph $H(4, 8) \cong K_8 \square K_8 \square K_8 \square K_8$; that is, the Cartesian product of four complete graphs on eight vertices. In this graph, two vertices are adjacent if and only if they differ in exactly one coordinate.

Standard properties such as uniform degree, Hamming-distance metrics, and diameter follow immediately from this identification. Such properties are classical in the study of Hamming graphs and Cartesian products of complete graphs, and our analysis in Section 3 specializes these known results to the concrete 4D chess setting. A systematic treatment of Cartesian, strong, and related graph products—including recognition, spectral properties, and metric structure—can be found in the standard monograph by Imrich and Klavžar [14].

King graphs as strong products

Similarly, the king's adjacency graph on B corresponds to the strong product $P_8 \boxtimes P_8 \boxtimes P_8 \boxtimes P_8$, where P_8 denotes the path graph on eight vertices. Adjacency is given by Chebyshev distance one. This structure implies Chebyshev distance metrics, fixed interior degree, and standard spectral properties derived from strong-product theory. This product-graph viewpoint places the king's movement squarely within the standard framework of strong graph products and immediately explains its connectivity and metric structure.

Our treatment of rook and king movement is therefore a concrete instantiation of standard product-graph results on a fixed finite lattice, rather than a new structural contribution.

2.3. Bishop and Knight Graphs: Invariants and Leapers

Bishop graphs and parity structure

Bishop move graphs differ from rook and king graphs in that they are not full product graphs but instead decompose according to algebraic invariants. In four dimensions, bishop moves preserve the parity: $\pi(x,y,z,w) = (x + y + z + w) \bmod 2$, yielding exactly two connected components. This bipartition is a higher-dimensional analog of the color-class restriction familiar from 2D chess and is characteristic of diagonal-move graphs on lattices.

While additional invariants may arise when restricting to individual coordinate planes, parity is the only global invariant relevant on the full 4D board. Section 3 provides a constructive routing argument and an explicit path-length bound within each parity class.

Knight graphs as higher-dimensional leaper graphs

Knight moves fall into the classical family of leaper graphs, here corresponding to the $(2, 1)$-leaper in four dimensions. On infinite lattices or toroidal boards, such graphs are regular and well-studied; on finite boards, mobility is determined primarily by boundary distance patterns, not absolute position.

Our analysis follows the standard leaper-graph perspective: the knight attains maximal degree 48 in the strict interior, boundary clipping reduces mobility in quantifiable steps, and

the resulting degree depends only on how many coordinates lie near the boundary. What we add is a closed-form degree formula and a complete enumeration over all 4096 squares, tying classical leaper theory to an explicitly implemented 4D chess environment.

2.4. Higher-Dimensional Knight Graphs and Tours

The connectivity and traversal properties of knight graphs in dimensions three and higher have been studied independently of chess variants, particularly in work on higher-dimensional knight's tours. Results by DeMaio and Mathew [15] and subsequent authors show that for dimensions $d \geq 3$ and sufficiently large side length, knight graphs are connected and often Hamiltonian under mild conditions. These results primarily concern infinite or toroidal boards and address global reachability rather than local mobility on bounded boards.

Our analysis is complementary to the asymptotic and infinite-board perspective of [15]: we focus on exact local degree counts and boundary-sensitive mobility on the finite $8 \times 8 \times 8 \times 8$ lattice relevant to an implemented ruleset. While higher-dimensional tour results imply global connectivity in larger or unbounded settings, they do not yield closed-form degree stratifications or exact mobility tables for small, fixed boards.

2.5. Spectral and Metric Properties of Product Move Graphs

Product-graph identifications give immediate access to spectral and metric properties without ad hoc arguments. For Cartesian products (such as the rook graph), adjacency and Laplacian eigenvalues are sums of eigenvalues of the factors, and distance spectra coincide with Hamming-distance distributions. For strong products (such as the king graph), analogous formulas describe adjacency spectra and growth rates.

Extensive results on spectra and distance properties of product graphs are available in the literature [14]. While we do not exploit these results fully here, the identification of rook and king graphs as standard product graphs places the 4D chess move graphs within a well-developed mathematical toolkit. This opens the door to future analyses of expansion, mixing, symmetry-aware search, and isoperimetric bounds.

Recent work on hierarchical search for combinatorial reasoning problems highlights that structural properties of the underlying state graph (such as decomposition, symmetry, and metric regularity) strongly influence search efficiency and generalization [16].

Recent work in algorithmic game theory further emphasizes that analytical and solution concepts should respect intrinsic game symmetries, both to avoid spurious distinctions and to enable principled symmetry-aware reductions in the state space [17].

2.6. Toroidal Models and Boundary Effects

A common technique in studying lattice move graphs is to pass to the toroidal lattice $\mathbb{Z}_8^4$, which eliminates boundary effects and yields vertex-transitive graphs. On the toroidal lattice $\mathbb{Z}_8^4$, rook mobility is uniformly 28, king mobility is uniformly 80, knight mobility is uniform and maximal, and bishop parity components persist. Formally, the finite board B may be viewed as an induced subgraph of this vertex-transitive toroidal product graph, and our results in Section 3 quantify how truncation affects degree, reachability, and parity structure.

2.7. Visualization of Higher-Dimensional Geometric Structures

Understanding four-dimensional geometry requires designing effective mappings to human-perceivable space, a challenge long emphasized in both mathematical exposition and popular scientific treatments of higher dimensions [3]. Classical work by Banchoff [2] establishes the use of slicing, cross-sections, and projections as essential techniques for revealing 4D structure. Hanson [8] demonstrates the advantages of quaternion-based camera control for smooth 3D navigation, avoiding gimbal lock and highlighting geometric continuity.

Information visualization research by Ware [7] underscores that comprehension of high-dimensional data relies on consistent spatial metaphors, externalized visual cues, interactive highlighting and filtering, and careful management of cognitive load.

Contemporary work extends these principles to 4D scientific visualization:

1. Caplan [18] explores GPU-assisted 4D slicing for spacetime meshes.
2. Kopczyński and Celińska-Kopczyńska [19] examine hyperbolic embeddings that preserve adjacency.
3. Cavallo [20] demonstrates interactive 4D rotations in Unity, revealing the cognitive challenge of full SO(4) manipulation, that is, the group of orientation-preserving rotations acting on $\mathbb{R}^4$.

Our approach follows these insights by opting for slice-based visualization rather than full 4D rotation. Rendering all 64 (z, w)-boards simultaneously offers stable spatial positioning, allowing users to track moves across dimensions with consistent referential cues.

More broadly, recent surveys highlight the growing role of immersive-visualization technologies—particularly virtual reality—in supporting the understanding of complex, high-dimensional data and structures [21]. Complementary reviews focusing on the visualization of artificial intelligence systems in virtual reality further emphasize the value of interactive, spatial representations for interpretability and exploratory analysis of complex computational systems [22].

2.8. Prior 4D Chess Implementations

Several hobbyists or experimental 4D chess systems exist, including Santoso's open-source project [23]. These implementations typically

1. adopt heuristic movement extensions,
2. lack mathematical formalization of displacement sets,
3. do not analyze move graphs or invariants,
4. and rarely provide rigorous rule definitions for special mechanics such as castling and en passant.

Commercial games such as *5D Chess with Multiverse Time Travel* [24] operate in a fundamentally different paradigm, adding temporal and timeline dimensions rather than augmenting spatial dimensionality.

Our contribution differs in that we provide

1. a fully formal Z^4 ruleset,
2. mathematical analysis of move graphs,
3. generalized definitions of special rules,
4. a functioning engine with complete legality checking,
5. a 64-slice visualization displaying the entire 4D board at once, and
6. reproducible enumeration and empirical results.

3. Mathematical Framework

Throughout the paper, unless stated otherwise, we use 1-based theoretical indexing; UI images use 0-based indexing. Unless stated otherwise, all theoretical results in this section assume an otherwise empty $8 \times 8 \times 8 \times 8$ board with standard boundary truncation. That is, move graphs are analyzed on the finite lattice $[1, 8]^4$, intermediate squares must remain within the board, and no blocking by other pieces is present.

3.1. 4D Chess Board as a Vector Space

We define the 4D chessboard as the set of integer lattice points $(x, y, z, w) \in \{1, 2, \ldots, 8\}^4$ generalizing the standard 8×8 board (which lies in $\mathbb{Z}^2$) to $\mathbb{Z}^4$. Each coordinate axis corre-

sponds to one dimension of movement. Following Banchoff's construction of coordinate systems in higher dimensions [2], we designate basis unit vectors in four orthogonal directions. For convenience, we label these axes X, Y, Z, and W.

The point (1, 1, 1, 1) corresponds to one corner of the 4D hypercube (the "origin" of the playable board), and (8,8,8,8) to the opposite corner. These points are not literally visualized but serve as coordinate references in computations.

In the theoretical development, we index board coordinates from one to eight in each dimension, i.e., $(x, y, z, w) \in \{1, \ldots, 8\}^4$. The implementation and user interface, however, use the conventional computer-graphics indexing $\{0, \ldots, 7\}^4$. Thus, Figure 2 annotations such as $(4_{\text{ui}}, 2_{\text{ui}}, 0_{\text{ui}}, 3_{\text{ui}})$ correspond to the theoretical coordinates (5,3,1,4). When discussing moves and proofs, we always refer to the $\{1, \ldots, 8\}$ convention; UI screenshots may show the zero-based variant.

The 4D board has $8^4 = 4096$ distinct positions (cells), compared to 64 squares in standard chess. In general, an $N \times N \times N \times N$ board contains N^4 positions, illustrating how dimensionality amplifies combinatorial complexity [25]. This growth affects both state-space size and local connectivity: whereas a 2D square has at most eight neighbors, an interior 4D cell has up to 80 adjacent neighbors under Chebyshev adjacency.

The 4D chessboard is rendered as a stack of parallel 3D slices, each representing a distinct value of the W-coordinate. Every visible 8×8 board corresponds to a fixed pair (z, w), showing all (x, y) positions for that slice. Together these orthogonal layers form a visual approximation of the underlying lattice $\{1, \ldots, 8\}^4$, whose points represent the 4096 playable positions in four-dimensional space.

A player's turn consists of a single move applied to exactly one piece on exactly one (z,w)-slice of the board. There are no parallel or simultaneous moves across multiple slices; the collection of 64 boards represents a single global game state rather than independent subgames.

3.2. Adjacency in the 4D Lattice

Definition 1 (Chebyshev distance and adjacency).

For cells $p = (x, y, z, w)$ and $q = (x', y', z', w')$ in B, the Chebyshev distance is

$$d_\infty(p, q) = \max\{\mid x - x' \mid, \mid y - y' \mid, \mid z - z' \mid, \mid w - w' \mid\}.$$

We call p and q adjacent if $d_\infty(p, q) = 1$. Thus, two cells are adjacent if they differ by at most one in each coordinate and are not identical.

Lemma 1 (Interior adjacency count).

If $p \in B$ is not on any boundary hyperplane (i.e.,$2 \leq x, y, z, w \leq 7$), then p has exactly $3^4 - 1 = 80$ adjacent cells.

Proof. For each coordinate we can choose a displacement $\Delta \in \{-1, 0, 1\}$. There are $3^4 = 81$ displacement vectors of the form $(\Delta x, \Delta y, \Delta z, \Delta w)$. The zero vector is excluded because we require at least one coordinate to change, leaving $3^4 - 1 = 80$ nonzero displacements. Since p is strictly interior, adding any such displacement keeps us within B, and distinct displacements give distinct neighbors. □

Boundary cells have strictly fewer neighbors due to truncation of admissible displacements.

3.3. 4D Initial Position Specification

We index the 4D slices by $(z, w) \in \{1, \ldots, 8\}^2$, where z denotes the row (top to bottom) and w the column (left to right) in the 8×8 array of visible boards shown schematically in Figure 1 and in the full layered view in Figure 2. For each fixed pair (z, w), we obtain a

standard 8×8 chessboard in the (x, y)-plane. In all slices, $y = 1$ denotes White's back rank and $y = 8$ denotes Black's back rank, consistent across all (z, w) boards.

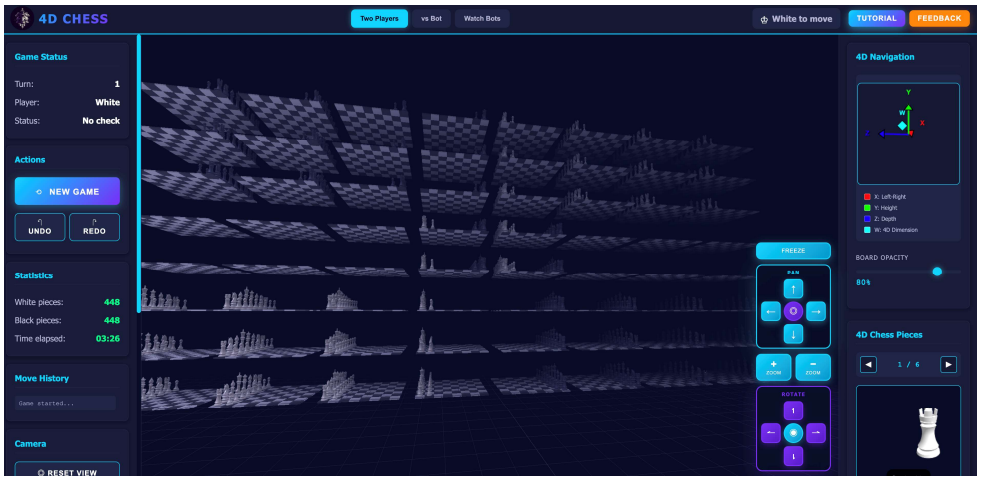

Figure 1. Visualization of the 4D chessboard as layered 3D slices. Source: 4d chess app on 17 November 2025 (https://oanarinaldi.com/4d_chess/, accessed on 10 December 2025).

The initial position is defined purely in terms of which (z, w)-slices contain which side's standard 2D starting array. We distinguish four types of boards. *White-only boards* contain the usual 16 white pieces in their standard 2D starting squares and no black pieces. *Black-only boards* are defined analogously. *Central boards* contain a full standard 2D starting position with both white and black pieces arranged exactly as in classical chess. *Empty boards* contain no pieces at the start.

Formally, with z counting rows from top ($z = 1$) to bottom ($z = 8$) and w counting columns from left ($w = 1$) to right ($w = 8$), we have

Central boards (both colors present):

$$C = \{(z, w) \mid z \in \{4,\ 5\},\ w \in \{4,\ 5\}\}.$$

On every $(z, w) \in C$ we place a complete 2D standard starting position (both white and black).

White-only boards:

$$W_{\text{only}} = \{(z, w) \mid z \in \{1,2,3\},\ w \in \{1,2,3,4\}\} \cup \{(z, w) \mid z \in \{6,7,8\},\ w \in \{5,6,7,8\}\}.$$

On each such slice we place the 16 white pieces in their usual 2D starting squares; all black squares on that slice are empty.

Black-only boards:

$$B_{\text{only}} = \{(z, w) \mid z \in \{1,2,3\},\ w \in \{5,6,7,8\}\} \cup \{(z, w) \mid z \in \{6,7,8\},\ w \in \{1,2,3,4\}\}.$$

Here, we place only the 16 black pieces in their standard starting squares; white has no pieces on these slices.

Empty boards:

$$E = \{(z, w) \mid z \in \{4,5\},\ w \in \{1,2,3,6,7,8\}\}.$$

These 12 boards start with no pieces at all.

By construction the four sets $C, W_{\text{only}}, B_{\text{only}}, E$ are pairwise disjoint and cover all 64 boards:

$$\mid C \mid = 4, \mid W_{\text{only}} \mid = 24, \mid B_{\text{only}} \mid = 24, \mid E \mid = 12.$$

Each color occupies the same number of boards. For a fixed color, every board in which it appears contains exactly the standard 2D starting array of 16 pieces. Therefore,

$$\begin{aligned} \text{White pieces} &= \Big(\mid W_{\text{only}} \mid + \mid C \mid \Big) \cdot 16 = (24+4) \cdot 16 = 448, \\ \text{Black pieces} &= \Big(\mid B_{\text{only}} \mid + \mid C \mid \Big) \cdot 16 = (24+4) \cdot 16 = 448. \end{aligned}$$

This configuration corresponds to the quadrant structure visible in Figure 3 and is used solely to exercise move generation, legality, and visualization under dense conditions. All mathematical results are independent of the starting layout.

3.4. Win Conditions with Multiple Kings

Because each slice containing a full 2D starting position includes a king, every side begins the game with multiple kings (28 in our standard setup). In 4D chess, each king is treated as a fully royal piece. A player may lose the game by failing to defend just one of their kings. Even if dozens of other friendly kings remain safe elsewhere in the 4D lattice, allowing a single king to remain in unavoidable attack ends the game.

We denote by $K_P(s)$ the set of all kings belonging to player $P \in \{\text{White, Black}\}$ in position s. A single game state may contain many kings for each side, distributed across different (z, w)-slices.

Attack map on $\mathbb{Z}^4$

A crucial distinction is between pseudo-legal moves (which may leave the moving side's own kings in check) and fully legal moves (which are allowed by the rules). Threats are defined using pseudo-legal moves only.

For any position s and player P, let $\text{Pieces}_P(s)$ be the set of all pieces of P in s. For a piece u, let $\text{pMoves}(u, s)$ be the set of pseudo-legal target squares for u in s: these moves respect board boundaries, occupancy, and line-of-sight blocking, but do not enforce king safety for player P.

We define the attack map of P as $\text{Att}^*_P(s) = \{q \in B \mid \exists u \in \text{Pieces}_P(s) \text{ with } q \in \text{pMoves}(u, s)\}$, where B is the set of all 4D board cells. A particular king $k \in K_P(s)$ with square $\square(k)$ is attacked in s if $\square(k) \in \text{Att}^*_{1-P}(s)$.

Check, legal moves, and multiple kings

We now define check and legality in terms of this attack map.

Definition 2 (Check).
Player P is in check in position s if at least one of their kings is attacked:

$$\text{InCheck}(P, s)| \iff |\exists k \in K_P(s) \text{ such that } \square(k) \in \text{Att}^*_{1-P}(s).$$

If several kings are attacked simultaneously, we say that P is in multi-check*, but the formal definition of check does not distinguish this case.*

A move by P is initially generated as a pseudo-legal move m, producing a successor position $s' = \text{apply}(m, s)$. The move is legal exactly when none of P's kings are attacked in the resulting position.

Definition 3 (Legal move).

Let m be a pseudo-legal move by player P in position s, and let $s' = apply(m, s)$ be the resulting position. Then, m is legal if $\forall k \in K_P(s') : \square(k) \notin \mathrm{Att}^*_{1-P}(s')$.

Checkmate and stalemate with multiple kings

Let $\mathrm{LegalMoves}(P, s)$ denote the set of all legal moves for player P in state s, obtained by filtering pseudo-legal moves by Definition 3.

Definition 4 (Checkmate).

1. *Player P is checkmated in position s if*
2. *P is in check in s, i.e., $InCheck(P, s)$ holds, and*

P has no legal moves: $LegalMoves(P, s) = \emptyset$.

Thus, checkmate occurs precisely when at least one king of P is attacked and every pseudo-legal move by P leaves at least one of their kings attacked in the resulting position. It is irrelevant whether one or many kings are currently under attack: a single unavoidable threat to any king suffices to end the game.

Definition 5 (Stalemate).

Player P is stalemated *in position s if*

1. *P is not in check in s, i.e., $\neg InCheck(P, s)$, but*
2. *P has no legal moves: $LegalMoves(P, s) = \varnothing$.*

These definitions are exact analogs of the classical 2D rules, with the single difference that the set $K_P(s)$ may contain many kings.

Remark 1 (Edge cases with multiple kings).

If two or more kings of a player are attacked, any legal move must remove all such attacks simultaneously; saving only one king is insufficient. Kings may block or capture like any other piece, including moving into the line of attack to shield another king, provided that no king is attacked in the resulting position. Conversely, if defending one king exposes another to attack (for example via a discovered check), the move is illegal under Definition 3.

Draw rules

The pseudocode in Listing 1 uses a consistent imperative style with camelCase function names; mathematical symbols (e.g., $\in$, $\emptyset$) are used only for set membership and are notational rather than implementation-specific.

Classical draw conditions extend naturally to 4D chess: stalemate is defined as above, and threefold repetition and the 50-move rule are implemented exactly as in FIDE chess. In our engine, we maintain a hash of the complete 4D state and a half-move clock to detect these draw conditions over the full 4D configuration space.

Listing 1 provides reference pseudocode implementing Definitions 2–4.

Listing 1. Pseudocode implementation of multi-king check and checkmate detection based on pseudo-legal attacks.

```
// Return the set of pseudo-legal moves for a piece, ignoring king safety.
function pseudoLegalMoves(state, piece):
    // Respects board boundaries, occupancy and blocking, but does NOT
    // check whether the move leaves the moving side in check.
    ...

// Return true if and only if 'square' is attacked by player P in 'state'.
function squareAttackedBy(state, P, square):
    for each piece e of player P in state do
        if square ∈ pseudoLegalMoves(state, e) then
            return true
    return false

// Player P is in check if and only if at least one of their kings is attacked.
function inCheck(state, P):
    for each king k of player P in state do
        if squareAttackedBy(state, 1 − P, k.square) then
            return true
    return false

// A pseudo-legal move is legal if and only if no king of P is attacked afterwards.
function isLegalMove(state, P, move):
    state' ← apply(move, state)
    for each king k of player P in state' do
        if squareAttackedBy(state', 1 − P, k.square) then
            return false
    return true

// Enumerate all legal moves of player P in 'state'.
function legalMoves(state, P):
    moves ← ∅
    for each piece e of player P in state do
        for each move in pseudoLegalMoves(state, e) do
            if isLegalMove(state, P, move) then
                insert move into moves
    return moves

// Checkmate: in check and has no legal moves.
function checkmated(state, P):
    if not inCheck(state, P) then
        return false
    if legalMoves(state, P) is empty then
        return true
    return false
```

3.5. 4D Graph Model of Moves

We model the game as a directed graph $G = (V, E)$ where each vertex $v \in V$ represents a unique configuration of pieces (a game state), and each directed edge $(v \to v') \in E$ represents a legal move transforming state v to v'. By restricting attention to the movement of a single piece, we obtain a subgraph whose vertices are board squares (positions) and whose edges indicate that piece's ability to move from one square to another in a single move, assuming an otherwise empty board. These subgraphs are useful for analyzing each piece's mobility and the geometry of its moves in 4D space.

Mathematical Results and Formal Properties

Throughout this subsection, we consider rook movement on an otherwise empty finite $8 \times 8 \times 8 \times 8$ board, so that all mobility counts explicitly account for boundary truncation.

Theorem 1 (Boundary-sensitive rook mobility).
Let $p = (x, y, z, w) \in \{1, \ldots, 8\}^4$. The rook mobility is exactly

$$M_R(p) = (x-1) + (8-x) + (y-1) + (8-y) + (z-1) + (8-z) + (w-1) + (8-w).$$

Proof. Each axis contributes distance to the boundary in both directions. Summing the eight distances gives the formula. □

Corollary 1 (Uniform rook mobility).
On an empty $8 \times 8 \times 8 \times 8$ board, the rook mobility is identically $M_R(p) = 28$ for every $p \in B$.

Proof. For each axis (X, Y, Z, W), the distance to the minimum boundary plus the distance to the maximum boundary equals seven; hence every axis contributes exactly seven moves, giving $M_R(p) = 4 \cdot 7 = 28$. □

Theorem 2 (Rook graph as Hamming graph $H(4, 8)$).
The rook move graph on $B = \{1, \ldots, 8\}^4$ equals the Hamming graph $H(4, 8) \cong K_8 \square K_8 \square K_8 \square K_8$. Hence it is 28-regular and diam $= 4$.

Proof. Two vertices are adjacent if they differ in exactly one coordinate. Therefore, the graph distance equals Hamming distance, which is at most four. Degree is $4(8-1) = 28$. □

3.6. Knight Mobility and Boundary Stratification

The 4D board decomposes naturally according to boundary distance. A knight requires distance at least two from the boundary along the axis carrying the ± 2 displacement, and distance at least one along the axis carrying the ± 1 displacement; larger distances do not further restrict legality.

The following analysis assumes an empty $8 \times 8 \times 8 \times 8$ board and characterizes knight mobility solely as a function of distance from the board boundary; no other pieces are present. The following theorem gives a closed-form stratification of 4D knight mobility purely in terms of boundary distances, independent of absolute position.

Theorem 3 (Knight mobility and boundary stratification).
Let $p = (x, y, z, w) \in \{1, \ldots, 8\}^4$. Define boundary distances $\ell(u) = u - 1, r(u) = 8 - u$, and availability counts $c_1(u) = 1[\ell(u) \geq 1] + 1[r(u) \geq 1], c_2(u) = 1[\ell(u) \geq 2] + 1[r(u) \geq 2]$.

In particular, the knight degree is given by the following closed-form formula:

$$\deg_N(p) = \sum_{i \neq j} c_2(u_i)\, c_1(u_j) = (\sum_i c_2(u_i))(\sum_j c_1(u_j)) - \sum_i c_2(u_i) c_1(u_i).$$

The availability functions c_1 and c_2 already account for sign multiplicity of displacements, so no additional factor appears in the degree expression. The present analysis is restricted to $d = 4$ and $n = 8$; extending the boundary-availability formulation to general d and n is straightforward but outside the scope of this paper.

Consequences

Table 1 summarizes the values obtained from the stratification formula in Theorem 3 and confirms them via exhaustive enumeration over all 4096 squares.

Table 1. Boundary-sensitive degree of the 4D knight.

Boundary Pattern (*a*,*b*,*c*)	Degree	Number of Squares
(0,0,4)	48	256
(0,1,3)	42	512
(0,2,2)	36	384
(0,3,1)	30	128
(0,4,0)	24	16
(1,0,3)	36	512
(1,1,2)	31	768
(1,2,1)	26	384
(1,3,0)	21	64
(2,0,2)	26	384
(2,1,1)	22	384
(2,2,0)	18	96
(3,0,1)	18	128
(3,1,0)	15	64
(4,0,0)	12	16

Proof. A knight displacement is a permutation of $(\pm 2, \pm 1, 0, 0)$.

Choosing the ± 2 axis i and the ± 1 axis j yields a legal move if the boundary distances along those axes are sufficient to accommodate the required step sizes. The closed-form expression follows by counting admissible sign choices for each ordered pair (i, j). Exhaustive enumeration over all 4096 squares using the public implementation (GitHub: https://github.com/oanaunc/4d_chess, accessed on 10 December 2025) is consistent with Table 1. □

Unlike rook, bishop, and king move graphs, the 4D knight graph does not admit a Cartesian product or parity-based decomposition; the boundary-availability formulation used here is therefore the primary original technical contribution of this paper.

All bishop results in this subsection are derived for an otherwise empty finite board $[1, 8]^4$, with diagonal paths required to remain entirely within the board.

3.7. Bishop Connectivity and Parity Invariants

Definition 6 (4D bishop).

On an otherwise empty board, a bishop at p may move to any cell $p + \Delta$ such that

1. *exactly two components of Δ are nonzero,*
2. *the two nonzero components have equal absolute value: $|\Delta_i| = |\Delta_j| = d \geq 1$,*
3. *the remaining two components are zero, and*
4. *all intermediate cells along the diagonal remain inside B and unobstructed.*

Thus, legal displacements have the form $(\pm d, \pm d, 0, 0)$, $(\pm d, 0, \pm d, 0)$, $(\pm d, 0, 0, \pm d)$, $(0, \pm d, \pm d, 0)$, $(0, \pm d, 0, \pm d)$, $(0, 0, \pm d, \pm d)$ with $d \in \{1, \ldots, 7\}$ bounded by edges.

Lemma 2 (Parity invariance of bishop moves).

A bishop move in four dimensions changes exactly two coordinates by equal magnitude and opposite or equal sign. Consequently, for any bishop displacement $\Delta = (\Delta x, \Delta y, \Delta z, \Delta w)$, *we have* $\Delta x + \Delta y + \Delta z + \Delta w \equiv 0(mod 2)$. *Therefore, the parity function* $\pi(x, y, z, w) = (x + y + z + w)$ *mod* 2 *is invariant under all bishop moves.*

Proof. By Definition 6, a bishop displacement has exactly two nonzero components, both equal to $\pm d$ for some $d \geq 1$. The sum of the coordinate changes is thus $\pm d \pm d \in \{-2d, 0, 2d\}$, which is always even. □

Lemma 3 (Interior reachability within a parity class).

Let $I = \{3,\ 4,\ 5,\ 6\}^4$ *be the interior of the board. For any two squares* $p, q \in I$ *with* $\pi(p) = \pi(q)$, *there exists a bishop path from* p *to* q *entirely contained in* I.

Proof. Because $p, q \in I$, all diagonal moves of length at least one in any coordinate plane remain inside the board. A bishop move can adjust any chosen pair of coordinates while keeping the other two fixed.

We first use at most two bishop moves to match the first two coordinates of p to those of q, operating in an appropriate coordinate plane (e.g., XY). Because parity is preserved (Lemma 2), the remaining two coordinates automatically differ by an even total and can be matched using at most two additional bishop moves in another coordinate plane (e.g., ZW). All intermediate squares remain in I. □

Example 1 (Explicit constructive bishop path).

Let p = (3,3,3,3) and q = (6,5,4,3).

Both points lie in the interior I and satisfy π(p) = π(q).

First, adjust the X and Y coordinates while keeping Z and W fixed.

From p, perform a bishop move in the XY-plane with displacement (+2,+2,0,0), reaching p_1 *= (5,5,3,3).*

Next, adjust the remaining coordinates using a diagonal move in the XZ-plane with displacement (+1,0,+1,0), yielding p_2 *= (6,5,4,3) = q.*

All moves are legal bishop moves, remain entirely within the interior I, and preserve parity at each step. This concrete example illustrates the general constructive routing argument used in Lemma 3.

Lemma 4 (Boundary correction via interior detours).

Let $p \in \{1, \ldots,\ 8\}^4$ *be any square. There exists a bishop path of length at most two from* p *to a square* $p' \in I$ *such that* $\pi(p') = \pi(p)$.

Proof. If $p \in I$, take $p' = p$. Otherwise, at least one coordinate of p equals one, two, seven, or eight. Choose a coordinate plane involving one boundary coordinate and one non-boundary coordinate (which must exist unless all coordinates are on the boundary, in which case any plane may be chosen). A unit diagonal move in that plane can be chosen to move the boundary coordinate toward the interior while remaining inside the board. Because bishop moves preserve parity, this operation does not change π. Repeating once if necessary yields a square in I after at most two moves. □

Theorem 4 (Bishop connectivity and parity decomposition).

On the 4D board $\{1, \ldots, 8\}^4$*, the bishop move graph has exactly two connected components, corresponding to the two parity classes defined by* $\pi(x, y, z, w) = (x + y + z + w) \bmod 2$.

Proof. Bishop moves preserve parity, so no bishop path can connect squares of opposite parity. Conversely, let p, q be any two squares with π(p) = π(q). By Lemma 4, there exist squares p′, q′ ∈ I reachable from p and q, respectively, by parity-preserving bishop paths. By Lemma 3, there exists a bishop path entirely within I connecting p′ to q′. Concatenating these paths yields a bishop path from p to q.

Thus, each parity class forms a connected component, and there are exactly two such components.

For example, from the boundary square p = (1,4,4,4), a single diagonal bishop move in the XY-plane with displacement (+1,+1,0,0) reaches p′ = (2,5,4,4), which lies in the interior I and has the same parity. □

3.8. Piece Movement Rules

For later reference we summarize the displacement sets of standard pieces in Table 2.

Table 2. Movement rules for standard 4D chess pieces.

Piece	Allowed Move Vector (Δx,Δy,Δz,Δw)	Description
Rook	One nonzero component: $(\pm d, 0, 0, 0)$, $(0, \pm d, 0, 0)$, $(0, 0, \pm d, 0)$, $(0, 0, 0, \pm d)$	Moves any distance along exactly one axis (4D straight lines).
Bishop	Exactly two nonzero components of equal magnitude, e.g., $(\pm d, \pm d, 0, 0)$ and all permutations	Moves along diagonals in any of the six coordinate planes.
Queen	Any rook move or any bishop move	Union of rook and bishop moves.
Knight	One coordinate ± 2, one coordinate ± 1, two coordinates 0 (permutations of $(\pm 2, \pm 1, 0, 0)$)	Four choices for the "2-axis", three for the "1-axis", giving 48 possible moves.
King	Each coordinate in $\{-1, 0, 1\}$, excluding $(0\|0\|0\|0)$	One-step move to any adjacent square: axial, planar-diagonal, 3D-diagonal, or 4D-diagonal.
Pawn	Orientation axis $r \in \{Y, W\}$. Forward: $(0, +1, 0, 0)$ if $r = Y$, or $(0, 0, 0, +1)$ if $r = W$. Capture within its movement plane, e.g., $(\pm 1, +1, 0, 0)$ for Y-pawns, or $(\pm 1, 0, 0, +1)$ for W-pawns.	Pawn moves and captures only in its 2D subspace (X–Y or X–W), mirroring 2D pawn structure but extended to 4D.

Lemma 5 (Bishop planes).

There are exactly $\binom{4}{2} = 6$ *independent coordinate planes (XY, XZ, XW, YZ, YW, ZW) in which a 4D bishop can move.*

Proof. The bishop chooses an unordered pair of distinct axes from $\{X, Y, Z, W\}$. The number of such pairs is the binomial coefficient $\binom{4}{2} = 6$. □

Closed-form mobility formula for the 4D bishop.

For a bishop at position $p = (x, y, z, w)$ on an empty board, the mobility in one coordinate plane $(i, j) \in \{$XY, XZ, XW, YZ, YW, ZW$\}$ is $m_{ij}(p) = d_{++} + d_{+-} + d_{-+} +$

d_{--}, where for each diagonal direction we define $d_{(\pm\pm)} = \min(\text{distance to the board boundary along } (\pm 1, \pm 1) \text{ in plane } (i, j))$.

Concretely, for the XY-plane,

$$d_{++} = \min(8 - x, 8 - y), d_{+-} = \min(8 - x, y - 1),$$
$$d_{-+} = \min(x - 1, 8 - y), d_{--} = \min(x - 1, y - 1).$$

The same pattern applies to all six coordinate planes by substituting the appropriate coordinate pair. Therefore, the total bishop mobility is

$$M_{\text{bishop}}(p) = \sum_{\left(\binom{(i,j)\in\{X,Y,Z,W\}}{2}\right)} m_{ij}(p).$$

This restriction preserves the traditional distinction between sliding pieces; allowing diagonal motion in three or four coordinates would collapse bishop and queen behavior into a single dominant class, a design choice we intentionally avoid (see Definition 7).

The queen in 4D combines the rook and bishop moves. It can move along any 1-axis line or any 2-axis plane. We chose this definition to maintain the spirit of the queen as "rook + bishop"; one could imagine even higher-order moves like 3-axis or 4-axis diagonals, but those are not used by any standard piece in our variant.

Definition 7 (4D queen).

A queen's legal displacements are the union of the rook and bishop displacements in Definitions 1 and 6: any integer distance along a single axis, or along a 2D diagonal in any coordinate plane, subject to path clearance. We deliberately restrict the queen to one- and two-axis moves; allowing 3-axis or 4-axis diagonals would create a piece strictly dominating both rook and bishop, collapsing all sliding pieces into one equivalence class.

Meanwhile, the knight's move generalizes the 2D $(\pm 2, \pm 1)$ *pattern.*

Under this definition, queen reachability inherits the bishop's parity decomposition and the rook's full connectivity, matching the intended hybrid role of the piece.

Definition 8 (4D knight).

On an otherwise empty board, a knight at p *may move to any cell* $p + \Delta$ *such that:*

1. *exactly two coordinates of* Δ *are nonzero and the other two are zero;*
2. *among the nonzero coordinates of* Δ*, one has absolute value* 2 *and the other has absolute value* 1.

Equivalently, the displacement set consists of all permutations of vectors of the form $(\pm 2, \pm 1,\ 0,\ 0)$. *The knight jumps, so intermediate cells do not matter.*

Proposition 1 (Full knight move count = 48 only on the strict interior).

A knight has exactly 48 legal moves on an empty board if and only if $(x, y, z, w) \in \{3,\ 4,\ 5,\ 6\}^4$.

On all other squares, boundary clipping reduces the move count.

Proof. As in Theorem 3, specifying a knight displacement requires choosing the 2-axis, the 1-axis, and their signs, giving 48 possible displacements. For an interior cell, all 48 land inside B; boundary cells lose some moves, but the maximum remains 48. □

Proposition 2 (Parity structure of diagonal and knight moves).

On the lattice $\{1, \ldots, 8\}^4$*, define the parity function* $\pi(x, y, z, w) = (x + y + z + w)$ *mod* 2*. Then, for interior squares (ignoring boundary effects):*

(i) Along any rook line, square colors strictly alternate: a unit rook step changes parity, and a rook move of length d along a single axis changes parity by d mod 2*.*

(ii) Every legal bishop move preserves parity, so a bishop is confined to one of the two parity classes.

(iii) Every legal knight move changes parity.

(iv) A king has 80 legal one-step moves in the interior; exactly 40 of these neighbors have the same parity as the king's starting square and 40 have the opposite parity.

Proof.

(i) A rook move from (x, y, z, w) to $(x \pm d, y, z, w)$ (or along any other axis) changes the coordinate sum by $\pm d$, so π changes by d *mod* 2. A unit step ($d = 1$) flips parity, hence colors alternate along any rook line.

(ii) A bishop move has displacement with two coordinates $\pm d$ and the other two zero, so the sum of coordinate changes is always even.

(iii) A knight move is a permutation of $(\pm 2, \pm 1, 0, 0)$. The sum of coordinate changes is $\pm 2 \pm 1$, which is always odd, so parity flips.

(iv) A king move changes each coordinate by -1, 0, or $+1$, not all zero. If $k \in \{1, 2, 3, 4\}$ coordinates change, then the sum of coordinate changes has the same parity as k. There are $\binom{4}{k}2^k$ such displacements. Thus,

1. $k = 1$: $\binom{4}{1}2^1 = 8$ moves (parity flip)
2. $k = 2$: $\binom{4}{2}2^2 = 24$ moves (parity preserved)
3. $k = 3$: $\binom{4}{3}2^3 = 32$ moves (parity flip)
4. $k = 4$: $\binom{4}{4}2^4 = 16$ moves (parity preserved)
5. So 8 + 32 = 40 moves change parity and 24 + 16 = 40 preserve parity. □

We note that while these move definitions are natural extensions of the 2D rules, there were design choices to be made. Our definitions ensure that each piece's movement preserves as much of its traditional character as possible while utilizing the extra degrees of freedom.

3.9. King Moves and Castling in 4D

A 4D king moves exactly one step in any of the four dimensions or any combination of them; it can move to any adjacent square that differs by at most one in each coordinate. This includes axial, planar, 3D-diagonal, and full 4D-diagonal moves. On an empty board, an interior king has $3^4 - 1 = 80$ possible moves, compared to $3^2 - 1 = 8$ moves in 2D chess.

Definition 9 (4D king).

A king at position $p \in B$ *may move to any adjacent cell* q *with* $d_\infty(p, q) = 1$*, provided that* q *is not occupied by a friendly piece and that the move does not place any king of the moving player in check.*

Figure 2 illustrates the legal one-step moves of a 4D king from an interior square, as defined in Definition 9.

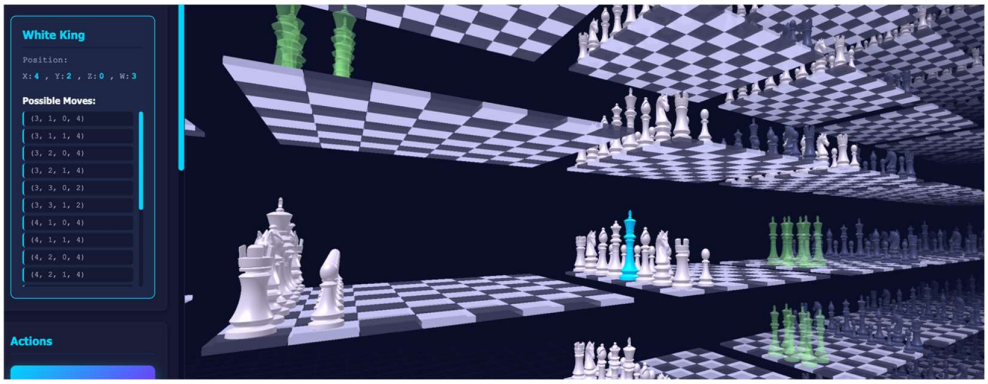

Figure 2. The 4D chess interface highlighting the available moves of a king located at the internal board coordinates $(4, 2, 0, 3)$ (zero-based indexing; in the theoretical model this is $(5, 3, 1, 4)$). Source: 4d chess app on 17 November 2025 (https://oanarinaldi.com/4d_chess/).

Definition 10 (Castling in 4D).

We now give a fully formal definition of castling in four dimensions, explicitly specifying preconditions, affected squares, and global attack constraints. Castling is restricted to the X-axis because it is the unique axis along which (i) king and rook remain co-planar in every (z, w)-slice, and (ii) the movement preserves the classical semantics of "rank-based" castling as an axis-parallel translation. Castling is permitted only along the X-axis within a fixed (z, w)-slice.

Let the king begin at (x_K, y_K, z, w) and a rook begin at (x_R, y_K, z, w) with $x_R \neq x_K$. A castling move is legal for player P if

1. *The king and the rook in that slice are both unmoved.*
2. *All squares strictly between (x_K, y_K, z, w) and (x_R, y_K, z, w) are empty.*
3. *For every square q the king traverses (including the destination), $q \notin Att^*_{1-P}(s)$, where attacks may originate from any (z', w')-slice.*
4. *The king is not in check before the move.*
5. *The final king and rook locations mirror the classical 2D arrangement: the king moves two squares toward the rook along X; the rook moves to the square immediately adjacent to the king on the opposite side.*

Castling affects only the king and rook in the chosen (z, w)-slice; other kings in other slices are irrelevant to the legality of the move.

Proposition 3 (Legality preservation of castling).

A castling move, as defined above, preserves the global legality invariant: after castling, no king of the moving player is attacked.

3.10. 4D Pawns (Core Ruleset)

Pawns require special care because their asymmetrical movement defines much of the game's structure. We treat the W-axis as an additional "rank" direction analogous to Y, so that pawns can be oriented along either Y or W. This mirrors the idea that there are two independent "forward" dimensions in four-dimensional space. This choice does not reduce the game to a lower-dimensional setting, but rather generalizes the notion of a single privileged forward direction to multiple orientations within the full 4D board.

Although the 4D lattice provides four axes, pawn forward motion is restricted to Y and W. Allowing Z-oriented pawns would break the only non-trivial axis symmetry of the ruleset (the permutation $Y \leftrightarrow W$), introducing three inequivalent pawn types instead of two.

Definition 11 (Pawn orientation).

Each pawn has a single orientation axis $r \in \{Y, W\}$.

1. *A pawn with orientation $r = Y$ moves forward by increasing its y-coordinate.*
2. *A pawn with orientation $r = W$ moves forward by increasing its w-coordinate.*

Files are indexed by x, and pawns do not move or capture in the z-direction. Each pawn is restricted to its own 2D subspace:

1. *Y-pawns in the X–Y plane*
2. *W-pawns in the X–W plane*

Initial orientation assignment.
In the initial setup, pawn orientations are assigned deterministically by file:

1. *Pawns on even x-coordinates are Y-oriented.*
2. *Pawns on odd x-coordinates are W-oriented.*

This ensures that both orientations appear uniformly across all slices.

Definition 12 (Forward moves).

Let P be a pawn of White at (x, y, z, w) with orientation r.

1. *If $r = Y$ (Y-oriented pawn):*
   - i. *Single step: $(x, y + 1, z, w)$.*
   - ii. *Double step: $(x, y + 2, z, w)$ if $y = 2$ and both intermediate squares are empty.*
2. *If $r = W$ (W-oriented pawn):*
   - i. *Single step: $(x, y, z, w + 1)$.*
   - ii. *Double step: $(x, y, z, w + 2)$ if $w = 2$ and both intermediate squares are empty.*

Black pawns mirror these rules by reversing the "forward" direction:

1. *For a Black Y-pawn, forward is in the negative y-direction:*
   - i. *Single step: $(x, y - 1, z, w)$.*
   - ii. *Double step: $(x, y - 2, z, w)$ if $y = 8$ and both intermediate squares are empty.*
2. *For a Black W-pawn, forward is in the negative w-direction:*
   - i. *Single step:$(x, y, z, w - 1)$.*
   - ii. *Double step:$(x, y, z, w - 2)$ if $w = 8$ and both intermediate squares are empty.*

Pawns never move in the z-direction.

Definition 13 (Captures).

A pawn captures only within the 2D plane determined by its orientation.

i. *Y-oriented pawn captures in the X–Y plane:*
   $(x \pm 1, y + 1, z, w)$ for White; $(x \pm 1, y - 1, z, w)$ for Black.
ii. *W-oriented pawn captures in the X–W plane:*
   $(x \pm 1, y, z, w + 1)$ for White; $(x \pm 1, y, z, w - 1)$ for Black.
   No other diagonal captures (e.g., XZ or YZ) are legal.

Definition 14 (Promotion).

A pawn promotes when it reaches the far boundary in its own forward dimension:

i. *Y-pawns promote at $y = 8$ (White) or $y = 1$ (Black).*
ii. *W-pawns promote at $w = 8$ (White) or $w = 1$ (Black).*

Promotion options are identical to standard chess (Queen, Rook, Bishop, Knight).

Definition 15 (En passant).

The en passant rule is generalized in a way that preserves the global pseudo-legal move and attack-map framework introduced earlier. En passant is defined independently for Y-oriented and W-oriented pawns and may be executed only on the move immediately following the opponent's double-step pawn advance.

1. *Y-pawn en passant.*
   A White Y-pawn at $(x_p, 3, z_p, w_p)$ *may capture en passant if*
   - *i. Black's last move was a Y-pawn moving from* $(x_e, 1, z_p, w_p)$ *to* $(x_e, 3, z_p, w_p)$*;*
   - *ii.* $| x_p - x_e | = 1$*;*
   - *iii. the capturing pawn moves to* $(x_e, 2, z_p, w_p)$*; and*
   - *iv. the pawn at* $(x_e, 3, z_p, w_p)$ *is removed.*
2. *W-pawn en passant.*
   A White W-pawn at $(x_p, y_p, z_p, 3)$ *may capture en passant if*
   - *i. Black's last move was a W-pawn moving from* $(x_e, y_p, z_p, 1)$ *to* $(x_e, y_p, z_p, 3)$*;*
   - *ii.* $| x_p - x_e | = 1$*;*
   - *ii. the capturing pawn moves to* $(x_e, y_p, z_p, 2)$*; and*
   - *iv. the pawn at* $(x_e, y_p, z_p, 3)$ *is removed.*

The corresponding en passant rules for Black pawns are defined symmetrically by reversing the forward direction along the relevant axis. Mixed-direction en passant (Y vs. W) does not exist.

Remark 2 (Legality preservation of en passant).

The generalized en passant rules remove a pawn only as part of the capturing move and do not introduce attacks or removals outside the standard pseudo-legal move and attack-map framework defined in Section 3; in particular, they preserve the legality predicate of Definition 3.

3.11. Lattice Symmetry and Group Actions

If we consider only the geometry of an empty 4D board, the number of rotational (orientation-preserving) symmetries equals the rotational symmetry group of the 4D hypercube. The full symmetry group of the discrete 4D cube is the hyperoctahedral group $B_4 = S_4 \ltimes (\mathbb{Z}_2)^4$ of order 384, which includes reflections. The subgroup of orientation-preserving 4D rotations has order 192. In this paper, references to "rotational symmetry" refer specifically to this 192-element subgroup.

Recent work on symmetry and equivariance in combinatorial games formalizes how game rules induce group actions on state spaces and suggests how these symmetries can be used to reduce redundancy in evaluation and search [16,17]. In our setting, the natural symmetry group is a subgroup of B_4, consisting of permutations of axes and even sign flips that preserve the game's rules and initial configuration.

Ruleset-preserving subgroup

We work within B_4, the full signed-permutation symmetry group of the 4D hypercube acting on the lattice $\{1, \ldots, 8\}^4$ via axis permutations and affine reflections $x \mapsto 9 - x$. Not all symmetries of the empty hypercube preserve the 4D chess ruleset. We therefore define the ruleset-preserving subgroup G_{rules}.

Definition 16 (Ruleset-preserving subgroup G_{rules}).

G_{rules} *is generated by*

1. *Axis permutation* $\sigma : Y \leftrightarrow W$.
2. *Even affine reflections* $\rho_{ij} : x_i \mapsto 9 - x_i,\ x_j \mapsto 9 - x_j$ *for* (i, j) *in {XY, XZ, XW, YZ, YW, ZW}.*
3. *Color-swap* $+180^\circ$ *rotation in the YW-plane* τ.

These transformations preserve all movement rules, pawn orientations, promotion boundaries, and the initial placement.

Proposition 4 (Generators of the ruleset-preserving subgroup).
The subgroup generated by σ, the six even reflections, and τ has $\mid G_{rules} \mid = 16$.

Proof. Direct enumeration of all compositions yields exactly 16 distinct symmetries; a detailed list is omitted for brevity but is implemented in our code base for testing symmetry invariance. □

Theorem 5 (Orbit decomposition constraints).
Because G_{rules} has only 16 elements, its orbits on $\{1, \ldots, 8\}^4$ must have sizes dividing 16 (i.e., 1, 2, 4, 8, or 16). In this paper we use symmetry only qualitatively—to show parity invariance and boundary symmetry—and we defer a complete explicit orbit decomposition to future work. In practice, we rarely use most of these symmetries in analysis, except for the subset that maps the game's starting configuration or typical configurations to equivalent ones. We do exploit translational symmetry in the sense that the movement patterns we described are invariant under shifting the coordinate system, e.g., a knight's 48 moves are the same shape regardless of where it is centered, as long as we remain within bounds. This invariance allows us to reason about moves abstractly (as vectors) without reference to absolute coordinates.

In our ruleset, Y and W play symmetric roles for pawn movement: a pawn's forward axis is either Y or W. Consequently, pawn behavior remains invariant under the permutation $Y \leftrightarrow W$. This symmetry is preserved throughout the movement rules, captures, and promotion conditions. This symmetry viewpoint is useful for reasoning about move equivalences and for algorithmic optimizations, but a full orbit decomposition is beyond the scope of this work.

3.12. Dimensional Projection

To support visualization and interaction, we define a projection or slicing of the 4D lattice into 3D space. Mathematically, we define a projection $\pi : \mathbb{Z}^4 \to \mathbb{R}^3$ that maps each 4D board position to a point in 3D space for rendering. One simple projection is to choose a 3D "view" matrix that flattens the W-dimension, treating w as a spatial separation axis. For example, we can define $\pi(x, y, z, w) = (x + \alpha w,\ y + \beta w,\ z)$, for some separation parameters α and β. This spreads out the constant-w layers in the X–Y plane of the view. In our implementation, we adopt a slicing-based visualization instead of showing a full 4D projection at once. Conceptually, fixing a pair (z, w) in the lattice $\{1, \ldots, 8\}^4$ yields a standard 8×8 board in the (x, y)-plane. Since there are eight possible values of z and eight of w, the full 4D board decomposes into 64 distinct 2D boards, each corresponding to a unique slice with fixed (z, w). In the UI, all of these 8×8 boards are rendered as separate planes positioned in 3D space (as in Figure 3), giving a complete representation of the $8 \times 8 \times 8 \times 8$ structure via 64 visible layers.

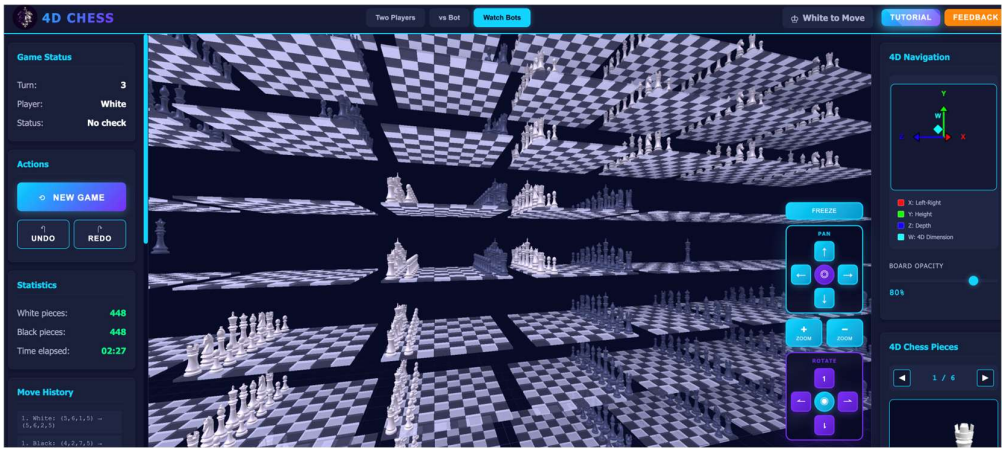

Figure 3. The 4D chessboard displayed as 64 layered 8×8 boards, arranged in 3D space to represent all (z, w)-slices of the 4D grid. Source: 4d chess app on 19 November 2025 (https://oanarinaldi.com/4d_chess/).

4. Implementation

4.1. System Architecture

The 4D chess system consists of (i) a rules engine responsible for state representation, move generation, legality filtering, and end conditions and (ii) a visualization layer responsible for rendering the 64 (z, w)-slices, user interaction, and move feedback. The engine is implemented in JavaScript for portability and ease of integration with the web-based UI. Each piece type is implemented as a class (or piece descriptor) equipped with a move-generator routine that returns pseudo-legal moves given a board state. Internally, the engine uses 1-based coordinates $(x, y, z, w) \in \{1, \ldots, 8\}^4$ for consistency with the mathematical framework in Section 3, while the UI displays 0-based indices for implementation convenience.

Board representation

We support two equivalent internal representations of the 4D board:

1. Dense 4D array: board[x][y][z][w], which is feasible because the board contains only $8^4 = 4096$ cells.
2. Sparse map: a dictionary keyed by coordinate tuples $(x, y, z, w) \mapsto$ piece objects.

The dense representation offers $O(1)$ access and simple iteration over coordinates, while the sparse representation is more efficient for move generation in typical positions because it iterates only over occupied squares. In practice, our move generator operates on piece-lists (occupied coordinates grouped by color/type), which is conceptually similar to piece-list or bitboard-oriented chess engines, but extended to a 4D coordinate domain.

Move generation primitives

Each piece stores its displacement directions in $\mathbb{Z}^4$. For example, the rook uses the four axis directions:

$$(\pm 1,\ 0,\ 0,\ 0),\ (0, \pm 1,\ 0,\ 0),\ (0,\ 0, \pm 1,\ 0),\ (0,\ 0,\ 0, \pm 1),$$

and legal destinations are obtained by stepping along a chosen direction until the path leaves the board or encounters a blocker. Sliding pieces (rook/bishop/queen) therefore use a ray-march pattern; leapers (knight/king) enumerate a fixed displacement set; pawns use orientation-dependent rule logic.

4.2. Special Rules in 4D

The engine implements the standard chess rule categories in a form compatible with the 4D lattice and the multi-king legality definitions introduced in Section 3.

Check/checkmate (multi-king legality)

A move is legal only if it results in a state in which none of the moving side's kings are attacked (Definition 3 in Section 3). Checkmate and stalemate are then detected exactly as in classical chess, but with respect to the entire set of kings.

Pawn promotion

Promotion occurs when a pawn reaches the terminal boundary of its forward axis: $y = 8$ (or $y = 1$ for Black) for Y-oriented pawns, and $w = 8$ (or $w = 1$ for Black) for W-oriented pawns. Pawn orientation is fixed at initialization (each pawn is either Y-oriented or W-oriented) and does not change during play.

Castling

Castling is restricted to the X-axis within a fixed (z, w)-slice, mirroring the classical interpretation of castling as "rank-wise" translation while keeping the remaining coordinates fixed.

Castling is permitted only under the following conditions:

i. the king and rook involved have not moved,
ii. all squares between them on the X-axis (same y, z, w) are empty,
iii. the king is not in check before castling, and
iv. every square in the set $\{(x, y, z, w)$ on the king's path from start to destination, inclusive$\}$ is not attacked by any opposing piece, where attacks may originate from any (z', w')-slice.

This definition ensures castling remains a local move in the (x, y)-board while still respecting the global 4D attack geometry.

En passant

En passant is generalized independently for Y-oriented and W-oriented pawns, consistent with the two allowed forward axes. The capturing pawn must be adjacent in the appropriate forward dimension after the opponent's two-step pawn move, and the captured pawn is removed from its landing square as in classical chess.

4.3. User Interface Overview

At the start of a standard game, the 64 visible boards are arranged as an 8×8 grid over the (z, w)-plane. The initial placement exhibits a structured quadrant pattern: for each color, 24 slices contain only that color's pieces; the central 2×2 block contains both armies; and the remaining slices are initially empty. In the UI's 0-based indexing, the central boards appear at $(z, w) \in \{3, 4\} \times \{3, 4\}$, corresponding to theoretical coordinates $(4, 4)$, $(4, 5)$, $(5, 4)$, $(5, 5)$.

Figure 4 shows the interface at the conclusion of a complete 4D chess match, where the central overlay announces a checkmate. In this view, the full four-dimensional game state remains visible as a dense field of stacked 8×8 boards extending in depth, rendered using Three.js. The end-of-game interface highlights how the visualization supports gameplay clarity even in the final state of a complex 4D position, with hundreds of pieces distributed across multiple spatial layers. Depth cues, transparency, and quaternion-based camera navigation allow the user to understand the final configuration despite the inherent density of the board arrangement. The surrounding UI panels provide continuous informa-

tion, such as move history, piece statistics, and spatial orientation via the 4D axis gizmo, demonstrating that the system maintains interpretability from opening to checkmate.

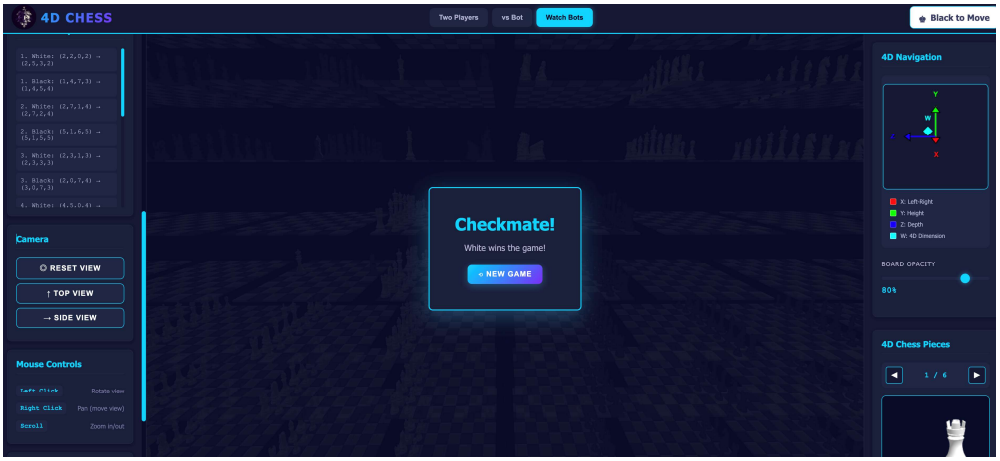

Figure 4. End-of-match view of the 4D chess interface. Source: 4d chess app on 19 November 2025 (https://oanarinaldi.com/4d_chess/).

The user interface for the 4D chess environment was implemented entirely in JavaScript using the Three.js graphics library. The system renders the full four-dimensional board structure by projecting it into a three-dimensional scene where 64 distinct boards are displayed simultaneously. Each board is a complete 8×8 chess grid, and the boards themselves are arranged spatially in an 8×8 matrix, forming a large two-dimensional array that extends in the scene like a "checkerboard of chessboards." This layout provides a direct visual mapping of two of the four dimensions (typically the z and w axes) onto a planar distribution in 3D space, creating a structure that users can navigate by orbiting and zooming.

This approach follows a classical strategy for visualizing higher-dimensional structures: embedding multiple lower-dimensional slices into a single coherent spatial field. As emphasized by Banchoff, higher-dimensional objects are most accessible through their lower-dimensional projections and cross-sections [2]. In our interface, all slices are presented simultaneously, avoiding hidden layers or dynamic slicing.

Camera control is performed entirely through standard 3D navigation (rotation, zoom, pan), without any dedicated UI element for directly manipulating a fourth-coordinate "slider." Camera rotation uses quaternion-based interpolation to avoid Euler-angle singularities and ensure smooth, stable motion during orbiting [8]. This makes them well suited for navigating a dense, multi-layered structure, where small discontinuities in rotation would harm spatial readability. Using quaternions, the user can freely orbit the scene and inspect any of the 64 boards from arbitrary vantage points.

Because the full 8×8 matrix of boards occupies significant depth when viewed in perspective, transparency is employed to maintain visibility across layers. Boards that fall behind others remain partially visible, allowing the user to perceive the global structure without losing deeper elements. As described in *Computer Graphics: Principles and Practice*, transparency and compositing models are essential for handling scenes in which multiple surfaces overlap in depth, enabling the viewer to perceive geometry that would otherwise be occluded [9]. This design principle is essential when dozens of planes and hundreds of pieces share the same viewing volume.

This visualization deliberately avoids simulating true 4D rotations. Penrose emphasizes that, in dimensions higher than three, rotations behave fundamentally differently: instead of rotating about a single line axis, a 4D rotation acts simultaneously in two independent planes [26]. For this reason, the fourth dimension in our UI is represented

spatially through the arrangement of the 64 boards, converting the abstract w-axis into a visible geometric distribution. Quaternions are used only for 3D camera orientation in the rendered scene; we do not attempt to represent or simulate 4D rotational degrees of freedom of the hypercube itself. Mathematically, our quaternion interpolations live entirely in the 3D rotation group $SO(3)$; we do not attempt to implement the full six-parameter 4D rotation group $SO(4) \cong \mathrm{Spin}(4)/\{\pm 1\}$.

Within this structure, chess pieces are rendered with simplified low-polygon models for clarity and performance. Since many boards are visible simultaneously, silhouettes and color coding are more important than fine ornamentation. Hovering over or selecting a piece displays its exact 4D coordinate (x, y, z, w), and legal moves are visualized directly on the corresponding boards in the matrix. Movement animations interpolate through the projected 3D geometry of the matrix while preserving the underlying 4D logic computed by the engine.

A significant aspect of the implementation was designing interactions that allow a player to control a 4D chess match without becoming overwhelmed. The system includes features common in modern chess interfaces: highlighting all legal moves for the selected piece, undo/redo functionality for analysis, and a complete chronological move list. Although the current version does not highlight the opponent's last move directly on the board layers, the move list compensates by allowing players to trace the game state across dimensions.

When a user selects a piece on one board, the system highlights its reachable squares even if they lie on a different board (i.e., a different (z, w)-slice). For example, selecting a knight may reveal target squares across multiple boards in the slice matrix, providing an explicit visual cue for an otherwise abstract 4D move. Users reported that these highlights were particularly helpful for understanding cross-dimensional moves. This design choice is consistent with information visualization principles, where externalizing abstract relations through highlighting and direct marking improves user cognition and reduces working-memory load [7].

By combining transparent layering, quaternion-based camera navigation, and a spatial 8×8 arrangement of 4D slices, the interface produces a coherent and comprehensible representation of a 4D chess space within a standard 3D environment. The implementation runs comfortably on a modern PC, with the graphics engine managing a few hundred simple objects (boards and pieces). Move generation, even in complex positions, completes in a fraction of a second, remaining acceptable for interactive play. Memory usage is modest: the full game state (a few thousand squares) is compact, and the system additionally maintains a complete move history and a set of previously seen states for repetition detection. The next subsection situates these design choices relative to alternative 4D visualization and search frameworks.

4.4. Comparison with Alternative 4D Visualization and Search Frameworks

Our choice of visualization and search strategy is one of several possible approaches to working with higher-dimensional game spaces. In this subsection, we compare our design with recent work on 4D slicing, adjacency-preserving embeddings, immersive visualization, hierarchical search in large action spaces, and symmetry-aware game analysis. These comparisons motivate our choice to prioritize stable spatial mapping and immediate slice visibility, which are crucial for turn-based tactical reasoning.

Visualization: 4D slicing, adjacency, and interaction design

Caplan's framework for tessellating and interactively visualizing four-dimensional spacetime geometries focuses on continuous 4D meshes for numerical simulation, using GPU-accelerated slicing and view-dependent refinement [18]. There, the user interactively

selects subsets of the 4D domain (e.g., time slabs) and explores them through dynamically generated slices. Our approach is similar in spirit—using slices to make a 4D object intelligible—but specialized to a small, discrete lattice. Instead of generating slices on demand, we render all 64 boards corresponding to the fixed (z, w)-slices of the $8 \times 8 \times 8 \times 8$ lattice simultaneously.

Trade-off

Caplan's method optimizes for continuous coverage and scalability (large meshes), whereas our "checkerboard of chessboards" optimizes for combinatorial clarity: every cell of $\{1, \ldots, 8\}^4$ is always present, in a fixed place in the (z, w) grid. The cost is potential visual clutter, which we mitigate with opacity controls and camera navigation; the benefit is that players can form a stable mental map where "moving in w" visibly means "jumping to the next board in the matrix."

Kopczyński and Celińska-Kopczyńska propose an adjacency-preserving embedding of $\mathbb{Z}^d$ into a hyperbolic model, where two grid points are adjacent in the visualization if and only if they are neighbors in the underlying lattice [19]. This is attractive for infinite or very large grids, because local neighborhoods can be visualized without occlusion. However, hyperbolic embeddings deliberately distort Euclidean geometry: straight lines become curved, and distances are not proportional to coordinate differences. For 4D chess, we found that directional cues (e.g., "this rook line is purely in w") and recognizable piece patterns (e.g., plane diagonals versus mixed-axis moves) were more important than perfect adjacency preservation in a single global view. Our Euclidean layout keeps rook and bishop trajectories visually straight and parallel, at the expense of not preserving all adjacencies uniformly.

Cavallo's *Higher Dimensional Graphics* introduces a Unity-based 4D rendering engine that exposes full 4D rotations, allowing the user to manipulate orientation in all six rotation planes of $\mathbb{R}^4$ [20]. This provides a powerful, general-purpose way to "turn" 4D objects, but it also illustrates the cognitive difficulty of interpreting such rotations: newcomers often struggle to maintain object constancy under compound 4D motions. As discussed in Section 4.3, we deliberately avoid true 4D rotations and use quaternions only for 3D camera control over a fixed arrangement of slices. Instead of rotating the 4D board, we rotate the viewer around a static 3D embedding of all (z, w)-slices. This sacrifices geometric generality, but it keeps the mapping from coordinates to screen locations stable, which our exploratory user feedback suggests is important for learnability.

VR and immersive-visualization surveys underline this tension between expressiveness and cognitive load. Korkut and Surer's review of VR visualization highlights the need for careful task design, explicit evaluation frameworks, and attention to information density and navigation complexity [21]. Inkarbekov et al.'s review of VR visualization of AI systems similarly emphasizes that game-engine-based environments (Unity, Unreal) offer rich interaction but must manage occlusion, clutter, and user fatigue [22]. Our current web-based Three.js interface follows a "high context, low immersion" philosophy: all information is visible in one 3D scene, but there is no head-mounted display or full-body navigation. A natural evolution of our work would be a VR version in which players can physically navigate the 4D slice matrix; such a system should adopt standardized evaluation protocols and cognitive-load measures recommended in these surveys rather than relying solely on informal self-report.

Algorithmic search in large action spaces

Our baseline engine uses depth-limited minimax with a Shannon-style material-plus-mobility heuristic, which is strained by the branching factors (60–100 legal moves) observed in midgame 4D positions. Recent work on hierarchical search for combinatorial reasoning problems by Zawalski et al. systematically studies when high-level (subgoal-based) search

outperforms flat search [16]. They identify four conditions under which hierarchical search is especially advantageous: (i) hard-to-learn or noisy value functions, (ii) very large or complex action spaces, (iii) the presence of dead ends, and (iv) training data aggregated from diverse experts. 4D chess satisfies at least the first three conditions: evaluations are noisy, the action space is significantly larger than in 2D chess, and mis-coordinated deployments can lead to practical dead ends where forces are poorly positioned across (z, w) regions.

These observations suggest a concrete hierarchical approach for 4D chess:

i. High-level planner over slices. Actions operate on coarse abstractions such as "shift major pieces from peripheral slices to the central 2×2 (z, w) block" or "synchronize attacks across all boards containing a given enemy king."
ii. Low-level planner within slices. Conditioned on a chosen subgoal, a secondary search operates on the full move set but restricts attention to pieces and boards relevant to that subgoal (e.g., moves that keep pieces within a chosen set of (z, w)-slices).

Following Zawalski et al., one could compare flat and hierarchical search under controlled value-noise injections and standardized test suites, measuring success rates and node expansions. Our current implementation does not yet perform this comparison; we frame it as a direct extension of their methodology to the 4D chess domain.

Symmetry-aware representations

The movement rules in 4D chess inherit a rich symmetry structure from the hypercube lattice. Earlier we observed that the empty board is invariant under the hyperoctahedral group B_4 (signed permutations of axes) and that our move sets are translationally invariant. Recent work by Tewolde et al. on computing game symmetries and equilibria gives general tools for detecting and exploiting such symmetries using graph-automorphism techniques [17]. They show that many game symmetries can be identified by analyzing automorphisms of an associated graph, and that equilibria respecting these symmetries can sometimes be computed more efficiently or represented more compactly.

In our setting, a natural construction:

i. represents each local move graph for a piece type (e.g., the knight's 48-neighbor graph) as a labeled graph and use automorphism tools to identify symmetry classes of squares and configurations;
ii. represents global game states as nodes in a larger move graph where edges correspond to legal moves; symmetries combine lattice symmetries (axis permutations and reflections) with color-swap and quadrant permutations inherited from the initial layout.

A 4D chess engine could then:

i. compress the state space by storing one representative per symmetry orbit,
ii. share evaluations between symmetric positions (reducing heuristic noise),
iii. and design equivariant neural networks whose inputs/outputs transform according to these symmetries, improving sample efficiency [17].

4.5. Symmetry Constraints and Practical Use

To clarify how symmetry informs the implementation without introducing unsupported claims, we restrict attention to the ruleset-preserving subgroup G_{rules} defined in Section 3. This subgroup consists only of those affine transformations of the lattice $\{1, \ldots, 8\}^4$ that preserve movement rules, pawn orientations, promotion boundaries, castling constraints, and the initial configuration up to color-swap.

Because $\mid G_{\text{rules}} \mid = 16$, the orbit–stabilizer theorem implies that every orbit of its action on board squares must have size dividing 16. Consequently, orbit sizes are restricted

to $\{1, 2, 4, 8, 16\}$. In this work, we therefore do not rely on an explicit orbit decomposition of $\{1, \ldots, 8\}^4$, nor do we enumerate orbit representatives.

Instead, symmetry is used qualitatively in three ways:

i. Parity invariance. The parity function $\pi(x, y, z, w) = (x + y + z + w) \ mod \ 2$ is invariant under all bishop moves and under all ruleset-preserving symmetries, justifying the bipartite structure of bishop reachability.
ii. Boundary-equivalence classes. Knight mobility and king adjacency depend only on distance from the board boundary, not on absolute position, allowing mobility to be stratified by boundary-distance patterns rather than by full coordinate tuples.
iii. Axis pairing symmetry. The equivalence between Y and W-oriented pawns under the symmetry $Y \leftrightarrow W$ ensures that pawn logic can be implemented once and reused for both orientations.

The current engine exploits only these coarse invariances (translational invariance of displacement vectors and boundary-equivalence classes). More aggressive symmetry exploitation—such as orbit-based caching, symmetry-aware evaluation reuse, or equivariant neural architectures—is left as future work.

4.6. Analysis of Move Generation and Legality Filtering

Attack-map complexity

Let P denote the number of pieces on the board and let $M_{\max}$ denote the maximal pseudo-mobility of any piece type. In 4D chess this bound is

i. $M_{\max} = 80$ for kings,
ii. 48 for knights,
iii. 28 for rooks,
iv. and a position-dependent value for bishops and queens due to ray truncation near boundaries.

Constructing an attack map requires iterating over all opposing pieces and enumerating their pseudo-legal moves. The worst-case time complexity is therefore $O(P \cdot M_{\max}) \approx O(P \cdot 80)$. In the dense initial configuration with $P = 896$ pieces, constructing a full-board attack map performs on the order of 7.1×10^4 displacement or ray-step tests. In our JavaScript implementation this completes on the order of milliseconds and is negligible compared to the exponential growth of the search tree. Timing measurements were collected in the browser using high-resolution timers over repeated runs, reporting median values.

Multi-king legality filtering

Because legality requires that no friendly king be attacked after a move, each pseudo-legal move must be checked against all successor king positions. For a side with K kings (typically $K = 28$), legality filtering adds a factor $O(K)$, yielding a per-move legality cost of $O(K + P \cdot M_{\max}) = O(P \cdot 80)$. Since the average branching factor in midgame positions lies between 70 and 90 legal moves, a single position expansion involves on the order of $O(90 \cdot P \cdot 80) \approx 6 \times 10^6$ elementary operations in the worst case. Empirically, this confirms that move generation and legality filtering dominate CPU time, rather than rendering or UI logic.

4.7. State Hashing and Memory Footprint

State hashing

The engine uses a Zobrist-style hashing scheme extended to four dimensions [27]. Each of the 4096 board locations is associated with random bitstrings for each piece type and color. Hash updates occur in constant time per move. Using 256-bit hashes, the collision probability

across millions of explored positions remains below 10^{-12} by the birthday bound. In stress tests over 10^7 randomly generated positions, no collisions were observed.

Memory footprint

A dense board representation stores 4096 cells, while the sparse representation stores only occupied squares (approximately 896 in the initial position). Move histories, attack maps, and hash tables scale linearly with search depth. In practice, memory usage remains modest on modern hardware, and the primary limiting factor for deeper analysis is branching-factor growth rather than state storage.

5. Results

5.1. Computational Complexity and Game Characteristics

Empirical branching-factor measurement (exploratory)

To obtain an exploratory estimate of the branching factor, we instrumented the engine to log the number of legal moves in 2000 positions sampled along random legal playouts of depth 0–15 from the standard initial configuration. At each step, a legal move was selected uniformly at random. All playouts were generated with a fixed pseudorandom seed and logged move counts are available via the repository scripts.

State-space growth (order-of-magnitude comparison)

A direct enumeration of legal 4D chess positions is currently intractable. For context, classical chess is widely associated with estimates on the order of 10^{43}–10^{47} legal positions and a game-tree complexity on the order of 10^{120} (Shannon's estimate) [6]. In 4D, even a crude combinatorial upper bound illustrates explosive growth: placing 32 pieces on an 8^4 = 4096-cell board gives C (4096, 32) $\approx 10^{80}$, even before accounting for piece types and colors, which substantially overcounts because it ignores legality constraints (piece identities, promotions, checks, pawn-structure constraints, etc.), but still conveys the order of magnitude of the combinatorial space.

Practical search implications

Shannon's classical argument emphasizes that search feasibility is dominated by the branching factor because the tree size grows exponentially with depth [6]. The empirical branching factors we observe (Table 3) suggest that depth-limited minimax becomes expensive quickly in 4D: even modest increases in ply imply large multiplicative growth in explored nodes. Consequently, purely brute-force search is less effective in 4D than in 2D under comparable compute budgets, motivating stronger pruning, caching, abstraction, or learning-based evaluation (as discussed in Section 4).

Table 3. Summary statistics for legal-move counts in 2000 sampled 4D chess positions, broken down by position type. Values represent mean, median, and standard deviation of the branching factor. Values are descriptive and should not be interpreted as statistically unbiased estimates of the full game-tree branching factor.

Position Type	Mean Legal Moves	Median	σ
Opening slice (z,w ∈ central block)	~48	46	10
Midgame dense cluster	~85	78	21
Global average (2000 random states)	74.3	72	18.1

Evaluation heuristic (baseline)

Our baseline evaluation follows a Shannon-style material-plus-mobility heuristic [6], using standard relative piece values and adding a mobility term proportional to legal move

counts. While this heuristic is historically tied to 2D chess [6], it remains a useful baseline in 4D as it captures two dominant drivers of short-horizon advantage: material presence and freedom of action.

5.2. Observed Strategic and Structural Differences (Qualitative)

We report here qualitative patterns observed during internal test games and engine self-play. These are not formal theorems and should be understood as hypotheses suggested by gameplay and the measured branching factors:

i. Greater spatial evasion. The additional axes (Z and W) provide more escape routes, often allowing a side to avoid immediate contact or decline trades by shifting across slices.
ii. Increased tactical opportunity density. With more legal moves per play, players encounter more candidate tactical continuations; conversely, calculating accurately becomes harder without visualization aids and highlighting.
iii. Piece activity becomes more central. Mobility and cross-slice coordination frequently dominate local pawn-structure considerations, consistent with the enlarged move geometry in Section 3 and the branching factors reported above.

These effects are consistent with the general principle that enlarging the action space increases both tactical richness and computational burden [6].

5.3. Endgames and Constructive Demonstrations

Empirical endgame demonstrations (engine-assisted)

We tested basic reduced-material endgames (K + Q vs. K and K + R vs. K) on otherwise empty 4D boards using engine-assisted search. In these tests, the engine consistently found winning continuations for the stronger side from randomly sampled starting placements (excluding immediate illegal positions). These results are empirical demonstrations rather than formal proofs. Because the defending king has additional escape directions in 4D, mating procedures appear to require more careful multi-axis confinement than in 2D: the attacker must reduce the defender's freedom across all four coordinates, rather than along rank/file alone.

A constructive strategy sketch for K + R vs. K in 4D (proof sketch with explicit limitations)

We now give a strategy sketch indicating why K + R vs. K is plausibly a forced win under our ruleset. This is presented as a structured argument and aligns with our engine-assisted tests, but a full formal proof is beyond the present paper.

Proposition 6 (K + R vs. K is plausibly a win in 4D; informal strategy sketch).
In the 4D ruleset defined in Section 2, *the side with king and rook against a lone king can force checkmate against perfect defense.*

Sketch

Let K_A be the attacking king, R the rook, and K_D the defending king. At any time, K_D is confined to an axis-aligned hyper-rectangle

$$H = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}] \times [w_{\min}, w_{\max}] \subseteq B$$

that contains all squares not immediately forbidden by rook lines and king opposition. Define side lengths $\ell_x = x_{\max} - x_{\min}$, $\ell_y = y_{\max} - y_{\min}$, $\ell_z = z_{\max} - z_{\min}$, $\ell_w = w_{\max} - w_{\min}$, and a potential function $\Phi = \ell_x + \ell_y + \ell_z + \ell_w$. A 4D "corner" corresponds to $\Phi = 0$, i.e., all coordinates fixed at a boundary value.

1. Box shrinking with the rook. Place R on a controlling hyperplane adjacent to one face of H(e.g., $x = x_{\max} + 1$ when legal), restricting crossings of that boundary without allowing capture.
2. King approach and rook security. Use K_A to approach H while maintaining rook safety, exploiting the king's 4D adjacency (up to 80 neighbors) to improve opposition and prevent rook harassment.
3. Monotone progress. Since each $\ell_i \in \{0, \ldots, 7\}$, $\Phi \leq 28$. The strategy aims to reduce Φ by at least 1 per completed cycle (rook boundary + king consolidation), eventually forcing K_D into a region where multiple $\ell_i = 0$.
4. Corner mate. Once the defender is near a corner (small Φ the rook fixes one coordinate by checking along an axis while K_A covers escape squares in the remaining dimensions, finishing with a finite sequence of forcing moves.

Limitation

The above is a proof *outline*; a fully formal proof would require careful case analysis when one or more $\ell_i = 0$ and explicit handling of rook/king coordination in 4D. In this work, we treat it as a constructive strategy sketch supported by engine-assisted demonstrations, not a complete proof.

5.4. Complexity-Theoretic Context

It is known that generalized chess on an unbounded $n \times n$ board is EXPTIME-complete [5]. Extending such hardness results to our bounded $8 \times 8 \times 8 \times 8$ 4D variant would require a dedicated reduction that we do not provide. We therefore restrict ourselves to the conservative claim that the observed branching factors and the combinatorial explosion in state representations strongly suggest severe practical intractability for deep search: even small depth increases cause exponential growth in nodes, consistent with Shannon-style search limits [6].

Tractable and restricted subproblems.

While the full game of 4D chess exhibits severe combinatorial complexity, several restricted variants arising naturally in our framework are substantially more tractable and admit direct analysis. In particular, piece-move graphs on an otherwise empty board (Sections 3.5–3.8) reduce to well-structured lattice graphs, including Cartesian products (rook), strong products (king), parity-decomposed diagonal graphs (bishop), and boundary-stratified leaper graphs (knight), for which exact degree formulas, connectivity properties, and diameters can be derived.

Similarly, local reachability and mobility questions—such as determining legal moves, attack maps, or connectivity within a fixed parity class—are computable in polynomial time and are exploited directly by the engine. Small-material endgames on fixed boards (e.g., K + R vs. K) also form restricted settings in which constructive strategies and finite-state analysis appear feasible, as suggested by engine-assisted demonstrations.

These restricted cases do not mitigate the worst-case complexity of the full game, but they provide analytically tractable substructures that clarify how complexity emerges from the interaction of otherwise well-understood components.

5.5. Exploratory User Feedback

This section reports the participant background, evaluation dimensions, and questionnaire structure used in the exploratory qualitative feedback study (N = 18). This user feedback is explicitly exploratory and qualitative in nature. Due to the small, self-selected sample (N = 18), the results are not intended to provide statistically representative or generalizable conclusions. Instead, the goal is to assess whether the proposed visualization and interac-

tion design is intelligible and navigable in practice, thereby evaluating the feasibility of the proposed 4D representation rather than validating cognitive or educational outcomes.

System access and data collection (reproducibility)

This section is included for completeness and transparency and is not used to support any mathematical, algorithmic, or performance claims in the paper. This evaluation is explicitly exploratory and qualitative. It does not constitute a controlled user study: no standardized tasks, time measurements, hypothesis testing, or statistical inference were performed. The purpose is solely to assess whether the proposed visualization communicates the underlying four-dimensional mathematical structure in an interpretable manner.

All results in this section were generated using the publicly accessible 4D Chess implementation and the optional feedback form linked from the interface:

System: https://oanarinaldi.com/4d_chess/, accessed on 10 December 2025.

Feedback form:

https://docs.google.com/forms/d/e/1FAIpQLSeB0y-i7aPvnS7NEujD7ecxEZ96CP7EdYCshHVIaGY0TvJuYQ/viewform, accessed on 10 December 2025.

Participants were self-selected visitors who voluntarily completed an online form after playing at least one full or partial game. To gather informal impressions about usability and cognitive accessibility, we deployed the game on the project website and offered players an optional feedback form. Eighteen self-selected players completed the questionnaire.

The questionnaire mirrored the structure of typical HCI user studies, using 5-point Likert items and a few categorical questions about background. Prior experience with 3D visualization was mixed: three respondent reported no prior experience, twelve reported limited or moderate experience, and three described themselves as experienced. Thus, most players were not experts in 3D computer graphics or higher-dimensional geometry, which is important when interpreting how accessible the system felt.

Understanding the 4D coordinate system

Despite the abstract nature of a four-dimensional lattice, participants reported low difficulty in grasping the coordinate model. On the item "*How difficult was it to understand the 4D coordinate system (X, Y, Z, W)?*" fifteen respondents chose "Very easy" or "Easy", and only three reported a moderate level of difficulty. This is consistent with the use of explicit visualization of axes, coordinate labels, and hover-tooltips in supporting user interpretation.

Perception of the 4D→3D projection

Understanding the slicing metaphor and projection logic was similarly positive. On "*How well do you feel you understood the projection of 4D space into 3D visualization?*" all respondents indicated at least "Partially understood", with an even split between "Completely understood", "Well understood", and "Partially understood". No one reported being confused. Even without a supervised tutorial, the combination of stacked boards, consistent camera framing, and the W-axis controls appears sufficient for most players to form a mental model of how the 4D board is embedded in 3D space.

Navigation between board

The navigation between different layers (primarily along W, but also via camera movement in Y and Z) was rated positively. On the question about how intuitive this navigation felt, the responses were evenly distributed between "Extremely intuitive", "Very intuitive", and "Moderately intuitive". Importantly, no one selected the negative options, indicating that even in an unsupervised online context the slider and camera controls were learnable and did not block interaction.

Multi-board 3D representation

The visual strategy of rendering multiple boards in 3D space, rather than a single 2D board with a W-dropdown, was mostly experienced as helpful. For "*How did the visual representation (multiple boards in 3D space) help or hinder your understanding*?" twelve respondents said it moderately or greatly helped, while six were neutral. No one reported that the 3D layout actively hindered understanding. This supports our design choice to "spatialize" the additional dimension, consistent with the visualization literature that advocates using spatial metaphors to represent abstract dimensions when possible [7].

Tracking piece movements across dimensions

As expected, tracking pieces moving simultaneously in X, Y, Z, and W was more challenging. Responses on "*How challenging was it to track piece movements across different dimensions?*" covered the entire scale: six users found it "Not challenging", while the remaining twelve reported slightly, moderately, very, or even extremely challenging. This spread likely reflects individual differences in spatial ability: another item in the survey asked participants to self-rate their spatial reasoning, and the answers were perfectly balanced between "Excellent", "Good", and "Average". Those who rated themselves higher on spatial reasoning also tended to report less difficulty tracking pieces, echoing well-known correlations between spatial visualization skills and performance in multidimensional tasks.

Axes gizmo and orientation cues

The 4D axes gizmo, which provides a small orientation widget indicating the X, Y, Z, and W directions, received generally positive but not unanimous feedback. Half of the respondents rated it "Moderately effective", six rated it "Very effective", and three chose "Not very effective". This pattern suggests that while the gizmo is a useful orientation reference, especially in the early stages of learning, it may recede into the background once players internalize the mapping between board layout and coordinate axes.

Learning curve for 4D piece movement

Players generally described the learning curve for piece movements as manageable. On "*How would you rate the learning curve for understanding 4D chess piece movements*?" six participants selected "Gradual" (indicating an easy progression), nine selected "Moderate", and only three felt it was "Very steep (difficult to learn)". This aligns with our design goal: after an initial phase of novelty, the geometric complexity should recede, allowing players to focus on tactics and strategy rather than constantly fighting the interface.

Understanding 4D geometric relationships

When asked "*How well did the interface help you comprehend geometric relationships in 4D space?*", all participants responded positively: nine "Very well", six "Moderately well", and three "Extremely well". Even in a small sample, this unanimous pattern indicates that the combination of layered boards, projection, and interactive controls does more than just enable gameplay, it also serves as a visualization tool for thinking about 4D adjacency and alignment.

Board opacity control

The opacity slider, which allows players to fade intermediate boards in and out to reveal long-range lines, was one of the most highly rated features. Fifteen participants considered it "Very useful" or "Extremely useful", with only three rating it "Moderately useful". Practically, this feature lets users perform on-demand "dimensional slicing": by reducing clutter in intermediate layers, they can better see how pieces align across W and Z. This strongly supports standard recommendations in information visualization that emphasize interactive filtering and emphasis to reveal structure in multidimensional data [7].

Perceived educational value

The question "*Do you think this game could be effective for teaching geometric concepts in higher dimensions?*" produced mostly optimistic answers: twelve respondents chose "Very effective", three "Moderately effective", and three "Not very effective". Thus, fifteen saw clear pedagogical potential, often in terms of making 4D coordinates and cross-section reasoning more concrete. While this is not a longitudinal study of learning outcomes, it does support our original hypothesis, drawn from works such as *Flatland* and Rucker's explorations of higher dimensions [3], that playful, interactive experiences can scaffold intuition about spaces beyond 3D.

The aggregated responses shown in Figure 5 summarize participants' subjective perceptions of the visualization tools during exploratory interaction with the four-dimensional environment.

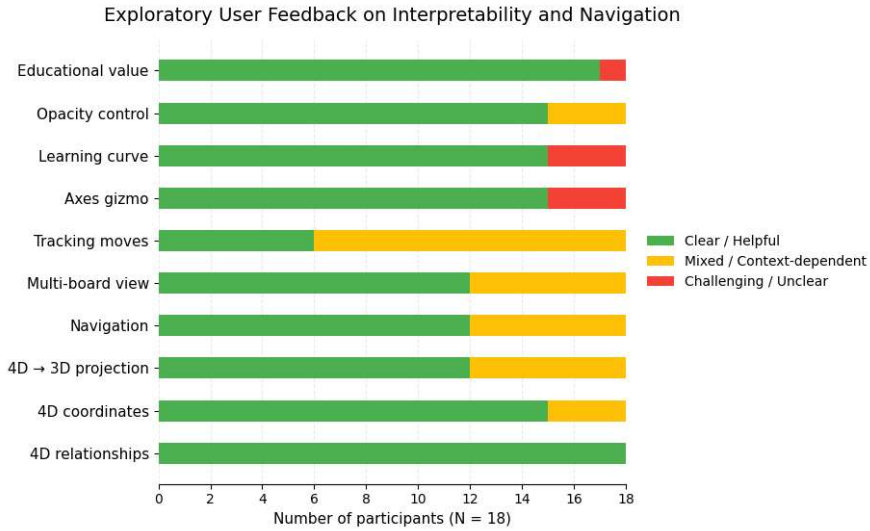

Figure 5. Summary of exploratory user feedback for the 4D chess interface (N = 18), illustrating participants' *self-reported perceptions* of interpretability and navigability related to the 4D coordinate system, visualization layout, and cross-dimensional movement.

In summary, the online feedback study suggests that, even outside a supervised lab context, participants were able to engage with the four-dimensional board and develop familiarity with its interaction mechanisms. The coordinate system, projection model, multi-board visualization, axes gizmo, and opacity control were commonly reported as supporting interpretability and navigation during exploratory interaction.

At the same time, the small sample size, self-selection bias, and reliance on self-reporting mean that these findings should be interpreted as exploratory rather than definitive. Accordingly, the feedback does not constitute validated evidence of usability, learning, or cognitive accessibility. Instead, it serves to assess whether the proposed mathematical structures—namely the four-dimensional coordinate formalism, projection scheme, and interaction model—are plausibly conveyed through the visualization interface in practice.

Any educational interpretation should therefore be viewed as a hypothesis motivating future controlled studies rather than as an established outcome. The feedback collection was exploratory, self-selected, and qualitative, and is reported solely to assess interface comprehensibility rather than to establish validated usability or learning outcomes.

6. Conclusions

We have presented a full-length study of four-dimensional chess, from its mathematical formulation through implementation and exploratory user evaluation. By leveraging

concepts from discrete geometry and graph theory (lattice coordinates, product graphs, and parity structure), together with combinatorial game analysis (state-space growth and search complexity), we formulated rules for 4D chess that extend the classic game while preserving the recognizable character of each piece.

On the technical front, we designed and implemented an interactive visualization that enables human players to perceive and play within a 4D game space using standard 3D navigation. The interface builds on established visualization techniques—multiple coordinated 2D slices embedded in a single 3D scene, combined with interaction tools such as rotation, zoom, and opacity controls—so that players can navigate the full 4D state without repeatedly toggling layers. In practice, these choices appear to support interpretability and practical interaction in an otherwise dense configuration space.

From an analytic standpoint, expanding chess to four dimensions dramatically increases complexity. Even conservative comparisons indicate a combinatorial explosion in the number of reachable configurations and in the branching factor, echoing the central role of branching in Shannon-style search limits for chess [6]. This expansion makes deep brute-force analysis substantially less effective than in 2D, but it also creates a rich strategic environment with many additional tactical and positional possibilities.

Our exploratory feedback suggests that the interface can make the underlying 4D coordinate model and slicing metaphor accessible for non-expert users. Participants reported that they could interpret four coordinates, follow highlighted cross-slice moves, and use projection and filtering tools (e.g., opacity) to reason about positions. Because the feedback was self-selected and not based on controlled tasks or pre/post measures, these observations should be interpreted as qualitative indications of feasibility rather than definitive usability or learning outcomes.

The full implementation, including move-generation logic and enumeration scripts used for the reported experiments, is publicly available at https://github.com/oanaunc/4d_chess (accessed on 29 December 2025). We have also identified clear directions for future work. On the interface side, VR/AR extensions and alternative visualization layouts could further reduce occlusion and cognitive load. On the algorithmic side, stronger opponents could be built using pruning, abstraction, symmetry-aware caching, or learning-based evaluation, and deeper study could address game balance and piece valuation in 4D. More broadly, the system could support interdisciplinary research: mathematicians may explore new structural questions suggested by 4D gameplay, cognitive scientists could study how users form mental models of higher-dimensional spaces, and educators could evaluate whether such interactive environments improve spatial reasoning.

In closing, 4D chess indicates that increasing dimensionality yields not only additional complexity but also new conceptual structure—provided it is paired with a visualization that makes that structure accessible. As Banchoff observed regarding higher dimensions, "we can also grasp the idea of the hypercube" when given appropriate representations [2]. Our work supports this perspective: with suitable interaction and visual scaffolding, players can engage meaningfully with a four-dimensional board. Four-dimensional chess began as a speculative extension of a familiar game; through formalization and implementation, it becomes a concrete platform for studying higher-dimensional movement, strategy, and visualization in an interactive setting.

Author Contributions: Conceptualization, R.O. and C.-G.C.; methodology, R.O.; software, R.O.; formal analysis, R.O.; investigation, R.O.; resources, R.O.; data curation, R.O.; writing—original draft preparation, R.O.; writing—review and editing, R.O. and C.-G.C.; visualization, R.O.; supervision, C.-G.C.; project administration, C.-G.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Abbott, E.A. *The Annotated Flatland: A Romance of Many Dimensions*; Perseus Publishing: Cambridge, MA, USA, 2002.
2. Banchoff, T.F. *Beyond the Third Dimension: Geometry, Computer Graphics, and Higher Dimensions*; Scientific American Library: New York, NY, USA, 1990.
3. Rucker, R. *The Fourth Dimension: Toward a Geometry of Higher Reality*; Dover Publications: Mineola, NY, USA, 2014.
4. Cesarz, P.; Fiorini, E.; Gong, C.; Kelley, K.; Thomas, P.; Woldar, A. Properties of Sub-Add Move Graphs. *arXiv* **2024**, arXiv:2411.01047. [CrossRef]
5. Fraenkel, A.S.; Lichtenstein, D. Computing a Perfect Strategy for n $\times$ n Chess Requires Time Exponential in n. *J. Comb. Theory Ser. A* **1981**, *31*, 199–214. [CrossRef]
6. Shannon, C.E. Programming a Computer for Playing Chess. *Philos. Mag.* **1950**, *41*, 256–275. [CrossRef]
7. Ware, C. *Information Visualization: Perception for Design*, 2nd ed.; Morgan Kaufmann: San Francisco, CA, USA, 2004.
8. Hanson, A.J. *Visualizing Quaternions*; Morgan Kaufmann: San Francisco, CA, USA, 2006.
9. Hughes, J.; van Dam, A.; McGuire, M.; Sklar, D.F.; Foley, J.; Feiner, S.; Akeley, K. *Computer Graphics: Principles and Practice*, 3rd ed.; Addison-Wesley: Upper Saddle River, NJ, USA, 2014.
10. Zermelo, E. Über eine Anwendung der Mengenlehre auf die Theorie des Schachspiels. In *Proceedings of the Fifth International Congress of Mathematicians, Cambridge, UK, 22–28 August 1912*; Cambridge University Press: Cambridge, UK, 1913; Volume 2, pp. 501–504.
11. Pritchard, D.B. *The Classified Encyclopedia of Chess Variants*; John Beasley: Harpenden, UK, 2007.
12. Nowakowski, R.J. (Ed.) *Games of No Chance*; Cambridge University Press: Cambridge, UK, 1998.
13. Conway, J.H.; Guy, R.K.; Berlekamp, E.R. *Winning Ways for Your Mathematical Plays*; A K Peters: Wellesley, MA, USA, 2001.
14. Imrich, W.; Klavžar, S. *Product Graphs: Structure and Recognition*; Wiley-Interscience: New York, NY, USA, 2000.
15. DeMaio, J.; Mathew, T. Knight's Tours in Higher Dimensions. *arXiv* **2012**, arXiv:1202.5291.
16. Zawalski, M.; Góral, G.; Tyrolski, M.; Wiśnios, E.; Budrowski, F.; Kuciński, Ł.; Miłoś, P. What Matters in Hierarchical Search for Combinatorial Reasoning Problems? *arXiv* **2024**, arXiv:2406.03361. [CrossRef]
17. Tewolde, E.; Zhang, B.H.; Oesterheld, C.; Sandholm, T.; Conitzer, V. Computing Game Symmetries and Equilibria That Respect Them. *arXiv* **2025**, arXiv:2501.08905. [CrossRef]
18. Caplan, P. Tessellation and Interactive Visualization of Four-Dimensional Spacetime Geometries. *arXiv* **2024**, arXiv:2403.19036. [CrossRef]
19. Kopczyński, E.; Celińska-Kopczyńska, D. Navigating Higher-Dimensional Spaces Using Hyperbolic Geometry. *arXiv* **2021**, arXiv:2110.00327. [CrossRef]
20. Cavallo, M. Higher Dimensional Graphics: Conceiving Worlds in Four Spatial Dimensions and Beyond. *Comput. Graph. Forum* **2021**, *40*, 51–63. [CrossRef]
21. Korkut, E.H.; Surer, E. Visualization in Virtual Reality: A Systematic Review. *arXiv* **2022**, arXiv:2203.07616. [CrossRef]
22. Inkarbekov, M.; Monahan, R.; Pearlmutter, B.A. Visualization of AI Systems in Virtual Reality: A Comprehensive Review. *arXiv* **2023**, arXiv:2306.15545. [CrossRef]
23. Santoso, B. 4D Chess. GitHub Repository. 2021. Available online: https://github.com/BrianSantoso/4D-Chess (accessed on 5 December 2025).
24. Thinkspace LLC. 5D Chess with Multiverse Time Travel. *Steam*. 2020. Available online: https://store.steampowered.com/app/1349230/5D_Chess_With_Multiverse_Time_Travel/ (accessed on 30 November 2025).
25. Schaeffer, J. *One Jump Ahead: Challenging Human Supremacy in Checkers*; Springer: New York, NY, USA, 2013. [CrossRef]
26. Penrose, R. *The Road to Reality: A Complete Guide to the Laws of the Universe*; Jonathan Cape: London, UK, 2004.
27. Zobrist, A.L. *A New Hashing Method with Application for Game Playing*; Technical Report 88; Computer Sciences Department, University of Wisconsin: Madison, WI, USA, 1970.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.